

Curated File Guide: script.js

The public website runtime that renders the portfolio, search, project windows, browser history, rich content, links, and AI chat behavior.

Before The Code: File Overview

Abstract

script.js is the public website runtime. It is the JavaScript that turns the static portfolio files into an interactive recruiter-facing site. The HTML page provides stable containers and the JSON catalog provides saved content, but script.js decides what appears, how projects are filtered, how rich text renders, how files become links, how project sections open into full-window views, how browser Back and Refresh interact with those views, how search behaves like a real search tool, and how the AI chat gathers useful context before asking the backend.

The important distinction is that script.js is public and read-only. It should never expose builder-only editing controls. Its job is to present what has already been saved and parsed by the builder. That is why much of the file is about rendering, normalization, routing, search indexing, assistant context selection, and UI state rather than file writes.

Introduction

When a visitor opens the portfolio, index.html loads styles.css, electronics-search.js, and script.js. script.js immediately grabs the DOM elements it will control: the project grid, search input, filters, profile fields, resume area, contact area, AI assistant, and footer. Then it waits for portfolio data. The data can come from an embedded window.__PORTFOLIO_CATALOG__ object in previews or from projects.json on the published website. Once the catalog is loaded, script.js normalizes profile data, renders the hero and contact information, creates category filters, renders project cards, builds the search index, and restores any deep-linked project section.

This file is the main reason the same portfolio can feel alive without requiring a database-backed website. It uses static files as the source of truth, then builds an application-like experience in the browser. That makes the public website easy to host on GitHub Pages or Cloudflare, while still giving visitors search, project windows, AI questions, and responsive navigation.

Background Information

The file works with a catalog generated by the builder parser. That catalog contains categories, projects, profile fields, site content, field styles, rich text blocks, fun facts, and optional site sections. The functions in script.js assume the catalog is already safe enough to display, but they still escape HTML where user-facing strings are inserted into the DOM. Rich text rendering is intentionally structured because portfolio content can include plain text, links, formulas, images, files, code blocks, captions, and nested project subsections.

The website also has a windowing model. Project cards appear inside categories. A project section button opens a full-screen dialog. Inside that dialog, subsections can be clicked like nested folders. Browser history is updated with the project id, section index, and resource path so the browser Back button can move through the same hierarchy. This makes project content feel more like a navigable portfolio than a long flat webpage.

Search is also broader than a normal text filter. The search index contains page sections, project names, category labels, tools, languages, rich text, files, URLs, captions, uploaded text files where fetchable, and electronics keyword expansions. The goal is that a recruiter can type an electronics phrase and land near the relevant artifact, even if the phrase appears inside a project section or file caption rather than on the top card.

Because the website is static, script.js must do work that a database-backed application might normally ask a server to do. It must decide which categories exist, which projects belong to each category, which sections are empty, which files should render as downloads, which links should open externally, and which project paths should be represented in browser history. The price of static hosting is that the browser runtime has to be disciplined. The benefit is that the public site remains easy to deploy, cache, mirror, and serve from GitHub Pages or Cloudflare without a long-running custom web server.

The file also sits between several other files. index.html gives it DOM anchors. styles.css gives visual meaning to the classes it writes. electronics-search.js can provide expanded electronics vocabulary. projects.json provides the portfolio catalog. The AI backend endpoint gives it remote intelligence when available. If one of those contracts changes, script.js is usually the first file that reveals the mismatch because rendering, search, or chat stops behaving correctly.

Purpose Of The File

The purpose of script.js is to convert portfolio data into a polished public experience. The file decides not only what to render, but also what not to render. Empty sections are hidden. Overview content appears as the first readable content when present. Links that look like websites open as websites rather than downloads. Local files become downloadable resources. Images render inline only when they are meant to be shown. AI links are only displayed when related to the question context.

Without script.js, index.html would be mostly a static shell with placeholders. There would be no dynamic project directory, no generated categories, no rich project windows, no browser-aware section routing, no searchable uploaded file text, no AI context gathering, no responsive project rendering, and no dynamic profile/contact population.

Primary Responsibilities

The first responsibility is state hydration. loadProjectCatalog reads the portfolio catalog, normalizes the data, stores it in module-level variables, and triggers rendering. This is the startup path for the public website.

The second responsibility is rendering. Functions such as renderProfile, renderSiteContent, renderFunFacts, renderSiteSections, projectCard, parsedSection, and parsedNodeContent turn structured catalog objects into actual HTML.

The third responsibility is interaction. The dialog functions create the full-window project section view, the routing functions keep browser history synchronized, the search functions build and display selectable results, and the AI functions manage the chat panel.

The fourth responsibility is interpretation. The assistant functions classify questions, gather relevant portfolio context, select public source links, decide whether a question is general or portfolio-specific, and prepare the request sent to the AI backend. This prevents the assistant from blindly dumping generic links for every question.

The fifth responsibility is restraint. Public visitors should see only finished, parsed, saved portfolio output. The file therefore hides empty content, avoids exposing builder edit controls, limits which sources are shown with AI answers, and uses normalized link behavior so a website link does not become a fake downloadable file. This restraint is what separates the public website from the local builder application.

The sixth responsibility is mobile and desktop continuity. The same project data must read cleanly on an iPhone and on a wide monitor. The responsive helper functions estimate project density and media presence, while the dialog and assistant functions keep windows and chat panels usable without letting one part of the page resize unrelated parts. script.js is therefore not only a renderer; it is also the public interaction policy for different devices.

Important Data Objects And Why They Matter

categories, projects, siteSections, profile, siteContent, funFacts, fieldStyles, and the rich variants of those fields are the hydrated catalog state. They are not hardcoded content. They are populated from projects.json or preview data, which is why the same script can run for Maurice's site and also for someone else's freshly installed builder output.

searchableEntries is the in-memory search index. Every item in it has a title, type, context, text, optional project id, optional section index, optional URL, and normalized text. The search functions score and display these entries.

assistantChatHistory and assistantSourceMap are the AI interface memory. assistantChatHistory keeps recent conversation so follow-up questions make sense. assistantSourceMap maps clickable source buttons in an answer back to portfolio search results so a visitor can jump to the relevant project or file.

currentSearchResults and searchResultLimit are the active search session. They remember what the latest query produced and how many items the user wants to see. Without these values, pressing Enter, selecting a result, or changing the result limit would not know which result list is current.

sectionRouteKey and sectionRouteParams are the browser-history contract for project windows. They keep the public URL connected to the currently opened project section and nested subsection. The values are small, but they are what let Refresh reopen the same project view and Back return to the previous view.

Top-Level Objects And Variables

This section explains the long-lived objects and variables before the function walkthrough. These are the values that give the file memory, configuration, API handles, DOM anchors, path boundaries, cache state, and behavior maps. They are explained by meaning, not by line number.

Dom Reference Or Ui State

grid

grid points at the project-grid container. renderProjects writes category sections and project cards into this element, making it the public project's visible mounting point. It anchors UI behavior to a specific browser element or window state. Functions that reference it include mediaGrid, renderSiteSections, parsedChildCards, projectCard, renderProjects, restoreSectionRouteAfterCatalogLoad, loadProjectCatalog.

searchInput

searchInput points at the project search box. Search handlers read its value, build dropdown results, apply highlights, and decide which projects remain visible. It anchors UI behavior to a specific browser element or window state. Functions that reference it include queueSearchHighlights, ensureSearchPanel, renderSearchResults, updateSearchDropdown, showSearchPanelIfNeeded, renderProjects, restoreSectionRouteAfterCatalogLoad.

filterButtons

filterButtons stores the current filter button elements. It is updated after dynamic filters are rendered because the original NodeList would not include newly created buttons. It anchors UI behavior to a specific browser element or window state. Functions that reference it include setActiveFilter, bindFilterButtons.

aiClearChatButton

aiClearChatButton is the explicit Clear chat control. It resets conversation history, source maps, and the visible log. It anchors UI behavior to a specific browser element or window state. Functions that reference it include updateAssistantPanelGrowth, restoreSectionRouteAfterCatalogLoad.

Portfolio Data State

projectFilters

projectFilters points at the category filter container. renderCategoryFilters fills it with filter buttons derived from the actual catalog categories. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderCategoryFilters.

projectCount

projectCount displays the number of visible or total projects. It is updated after catalog load and render operations so the summary band matches the data. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderProjects.

projectTrackCount

projectTrackCount displays how many project categories/tracks exist. It is derived from the saved categories rather than hardcoded. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderCategoryFilters.

projectTrackLabels

projectTrackLabels displays the category names. It helps the top summary reflect Analog, Digital, Embedded, or any custom categories the builder creates. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderCategoryFilters.

resumeSection

resumeSection is the public resume panel. renderProfile hides or shows it depending on whether a resume URL exists. Functions that reference it include renderProfile.

profilePhoto

profilePhoto is the large contact-section profile image. It is separate from the header avatar so desktop/mobile layouts can size them differently. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderProfile.

projects

projects is the runtime project array. Rendering, search, assistant context, and project windows all read from it. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include addSiteSectionSearchEntries, rebuildSearchIndex, assistantNamedProjectIds, assistantQuestionHasPortfolioIntent, assistantQuestionTokens, assistantNamedProjectTopicTokens, assistantContextForQuestion, assistantFallbackResults, assistantLocalAnswer, sectionRouteUrl.

siteSections

siteSections stores custom main-page sections such as personal profile, hobbies, sports, social media, or professional material. Functions that reference it include addSiteSectionSearchEntries, assistantPublicProfileLinks, assistantContextForQuestion, renderSiteSections, loadProjectCatalog.

profile

profile stores owner identity: name, email, phone, images, resume, GitHub, LinkedIn, and other public contact details. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderProfile, isWebsiteLinkItem, projectResponsiveClass, responsiveStyleValues, addSiteSectionSearchEntries, assistantUrlsFromTextValue, assistantPublicProfileLinks, assistantQuestionAllowsPublicLookup, assistantContextForQuestion, assistantFallbackResults.

profileRich

profileRich stores rich versions of profile/contact fields so styled text can survive from builder to website. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderProfile, loadProjectCatalog.

activeSectionDialogDrag

activeSectionDialogDrag stores the drag state while a project window is being moved. It anchors UI behavior to a specific browser element or window state. Functions that reference it include beginSectionDialogDrag, moveSectionDialogDrag, endSectionDialogDrag.

activeSectionDialogResize

activeSectionDialogResize stores the resize state while a project window edge is being dragged. It anchors UI behavior to a specific browser element or window state. Functions that reference it include beginSectionDialogResize, moveSectionDialogResize, endSectionDialogResize.

sectionDialogDragEnabled

sectionDialogDragEnabled prevents drag handlers from being installed more than once. It anchors UI behavior to a specific browser element or window state. Functions that reference it include enableSectionDialogDrag.

isApplyingSectionRoute

isApplyingSectionRoute prevents browser history updates from recursively triggering themselves while a saved route is being restored. Functions that reference it include syncSectionRouteToHistory, clearSectionRouteInHistory, applySectionRoute.

sectionRouteKey

sectionRouteKey identifies portfolio section entries in browser history state. Functions that reference it include sectionRouteState, sectionRouteFromState.

sectionRouteParams

sectionRouteParams defines the URL parameter names for project id, section index, and nested subsection path. Visible object fields include path, project, section. Those fields form the contract consumed by related functions. Functions that reference it include sectionBaseUrl, sectionRouteUrl, sectionRouteFromLocation.

defaultProfile

defaultProfile is the empty-owner fallback. It defines all expected profile fields so renderProfile can run even before the user enters contact details. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Visible object fields include brandImage, brandText, contactIntro, displayName, email, githubUrl, herolImage, linkedinUrl, phone, portfolioLabel, profileImage, resumeUrl, websiteUrl. Those fields form the contract consumed by related functions. Functions that reference it include normalizeProfile.

Temporary Calculation

year

year points at the footer year span. Startup code can set it from the current date so the footer remains current without manual HTML edits. Functions that reference it include renderProfile.

funFactsCallout

funFactsCallout is the near-top personality callout. renderFunFacts only reveals it when saved fun facts exist. Functions that reference it include renderFunFacts.

heroEyebrow

heroEyebrow is the small label above the hero title. renderSiteContent can replace it from the builder's front page settings. Functions that reference it include normalizeSiteContent, renderSiteContent, addSiteSectionSearchEntries.

heroTitle

heroTitle is the main public headline. It receives the owner-controlled front-page title instead of forcing a fixed sentence into every portfolio. Functions that reference it include normalizeSiteContent, renderSiteContent, addSiteSectionSearchEntries.

heroCopy

heroCopy is the supporting paragraph in the hero section. It is one of the fields affected by rich text and plain text styling choices. Functions that reference it include normalizeSiteContent, renderSiteContent, addSiteSectionSearchEntries.

brandName

brandName is the bold brand text in the header. renderProfile updates it to match the portfolio owner or site title. Functions that reference it include renderProfile.

brandSubtitle

brandSubtitle is the small subtitle beside the brand. It usually communicates local builder/public portfolio context or a tagline. Functions that reference it include renderProfile.

brandIcon

brandIcon is the image element for the OMB visual mark. Profile settings can change it when the builder is generalized for other users. Functions that reference it include renderProfile.

brandText

brandText is the OMB engraving text in the header logo area. It carries the small trademark-like label. Functions that reference it include normalizeProfile, renderProfile.

headerAvatar

headerAvatar is the clickable top-right profile image container. renderProfile reveals it only when a profile image exists. Functions that reference it include renderProfile.

headerAvatarImage

headerAvatarImage is the actual img element inside the avatar link. It receives the saved profile picture URL. Functions that reference it include renderProfile.

heroImage

heroImage is the main hero/background image element. Site content settings decide whether it is displayed. Functions that reference it include normalizeProfile, renderProfile.

contactBand

contactBand is the final dark contact area. It also contains the AI assistant panel, so layout code must keep chat growth from distorting contact information. Functions that reference it include renderProfile.

contactTitle

contactTitle is the heading for the contact panel. renderProfile updates it from the owner identity. Functions that reference it include renderProfile.

contactIntro

contactIntro is the contact-section introduction text. It can be empty, plain text, or rich content depending on saved profile data. Functions that reference it include normalizeProfile, renderProfile, addSiteSectionSearchEntries.

contactDetails

contactDetails is the list of direct contact methods such as email and phone. renderProfile builds clickable mailto and tel links inside it. Functions that reference it include renderProfile.

contactLinks

contactLinks is the container for external profile links such as GitHub, LinkedIn, resume, or custom websites. Functions that reference it include renderProfile.

footerOwner

footerOwner is the footer ownership text. It lets the footer follow the current profile owner rather than remaining generic. Functions that reference it include renderProfile.

searchWrap

searchWrap is the parent around the search box. The search dropdown attaches near it so results appear where the user is typing. Functions that reference it include ensureSearchPanel.

categories

categories is the runtime copy of project categories loaded from the catalog. It can include more than the original Analog, Digital, and Embedded groups. Functions that reference it include slugLabel, setActiveFilter,

renderCategoryFilters, addSiteSectionSearchEntries, assistantContextForQuestion, assistantFallbackResults, projectCard, renderProjects, openParsedSection, loadProjectCatalog.

funFacts

funFacts stores simple fun fact strings. renderFunFacts turns at most a few lines into a near-top callout. Functions that reference it include renderFunFacts, loadProjectCatalog.

funFactsRich

funFactsRich stores rich-format fun facts when the builder saves styled content instead of plain strings. Functions that reference it include renderFunFacts, loadProjectCatalog.

siteContent

siteContent stores front-page text and site-level settings such as hero copy and titles. Functions that reference it include renderSiteContent, loadProjectCatalog.

siteContentRich

siteContentRich stores rich versions of site-level text fields. Functions that reference it include renderSiteContent, loadProjectCatalog.

activeFilter

activeFilter stores the currently selected project category filter. Functions that reference it include projectVisible, setActiveFilter, renderCategoryFilters, renderProjects.

searchPanel

searchPanel stores the dropdown result panel created for search. Functions that reference it include ensureSearchPanel, renderSearchResults, goToSearchResult, restoreSectionRouteAfterCatalogLoad.

searchStatus

searchStatus stores the status element inside the search dropdown. Functions that reference it include ensureSearchPanel, setSearchMessage.

searchLimitSelect

searchLimitSelect stores the dropdown control that lets the visitor choose 5, 10, 15, or 20 results. Functions that reference it include ensureSearchPanel.

currentSearchResults

currentSearchResults is the current ranked search-result array. It lets click handlers navigate results without recalculating the search. Functions that reference it include ensureSearchPanel, renderSearchResults, restoreSectionRouteAfterCatalogLoad.

searchableEntries

searchableEntries is the full in-browser search index. It contains project titles, sections, files, captions, page sections, uploaded text, and electronics keyword expansions. Functions that reference it include rebuildSearchIndex, searchResultsFor.

searchResultLimit

searchResultLimit stores how many dropdown results to display. Functions that reference it include ensureSearchPanel, renderSearchResults.

searchHighlightTimer

searchHighlightTimer debounces highlight updates so typing does not repaint the page too aggressively. Functions that reference it include queueSearchHighlights.

searchHighlightName

searchHighlightName is the CSS class applied to highlighted search matches. Functions that reference it include clearSearchHighlights, applySearchHighlights.

defaultSiteContent

defaultSiteContent is the fallback front-page copy for a blank install. It keeps the site readable before the builder user customizes hero text and site messaging. Visible object fields include heroCopy, heroEyebrow, heroTitle. Those fields form the contract consumed by related functions. Functions that reference it include normalizeSiteContent.

Ai Configuration Or Conversation State

aiAssistantForm

aiAssistantForm is the chat form. Its submit handler starts answerAssistantQuestion. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include updateAssistantPanelGrowth, setAssistantBusy, restoreSectionRouteAfterCatalogLoad.

aiAssistantPanel

aiAssistantPanel is the whole Ask My Portfolio panel. Growth and layout code use it to keep the assistant right-justified and scrollable. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include assistantDesktopDockedHeight, updateAssistantPanelGrowth.

aiAssistantInput

aiAssistantInput is the question field. It is cleared/focused by chat handlers and intentionally does not keep long instructional text inside the typed value. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include setAssistantBusy, clearAssistantChat, answerAssistantQuestion, restoreSectionRouteAfterCatalogLoad.

aiAssistantLog

aiAssistantLog is the scrollable message history. appendAssistantMessage and replaceAssistantMessage write user and assistant messages into it. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include updateAssistantPanelGrowth, appendAssistantMessage, scrollAssistantAnswerToStart, clearAssistantChat, restoreSectionRouteAfterCatalogLoad.

aiAssistantStatus

aiAssistantStatus is the small status line used for ready/thinking/error states. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include setAssistantStatus.

assistantSourceCounter

assistantSourceCounter creates stable ids for assistant source buttons. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include assistantSourcesMarkup, clearAssistantChat.

assistantSourceMap

assistantSourceMap connects source button ids to source records so clicking a source opens the right place. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include assistantSourcesMarkup, clearAssistantChat, restoreSectionRouteAfterCatalogLoad.

Appearance Configuration

fieldStyles

fieldStyles stores plain-field font, size, and color choices. It lets front-page, profile, and contact fields render consistently with builder formatting. Functions that reference it include applyPlainFieldStyle, plainFieldStyleAttribute, loadProjectCatalog.

legacyTemplateSkins

legacyTemplateSkins maps older template ids to the newer appearance-only template ids. It keeps older saved projects rendering after template behavior changed. Visible object fields include skin-light-blue, skin-clean-white, skin-deep-navy, skin-red-warm, skin-emerald-instrument, skin-amber-power, skin-violet-mixed, skin-graphite-asic, analog-opamp-topology, analog-power-charger, analog-filter-front-end, analog-sensor-interface, analog-mixed-signal-timing, digital-fpga-pipeline, digital-asic-block, digital-verification-suite, digital-interface-controller, digital-signal-processing. Those fields form the contract consumed by related functions. Functions that reference it include canonicalTemplateId.

Runtime State Or Cache

assistantChatHistory

assistantChatHistory stores recent user and assistant messages. It gives the chatbot conversation memory. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include assistantRemember, clearAssistantChat, answerAssistantQuestion.

Compiler Or Language Configuration

supportedCodeLanguages

supportedCodeLanguages is the public language list for code rendering and detection. It names C, C++, Verilog/SystemVerilog, Java, JavaScript, Python, HTML, and related aliases. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Functions that reference it include normalizeCodeLanguage, codeLanguageLabel.

Code Walkthrough

This walkthrough is function-by-function. Related ideas are mentioned inside each function's explanation, but the headings are the actual function names so the document can be used as a code-reading companion even as line numbers change.

Public runtime utility

The functions in this group all support public runtime utility. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function normalize(value)

Purpose and intuition: normalize standardizes value data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `normalize` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `normalize`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function `sourceLooksCpp(source = "")`

Purpose and intuition: `sourceLooksCpp` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `sourceLooksCpp` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `sourceLooksCpp`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

text: `text` stores a computed value for temporary calculation. It is initialized from `String(source || "")`. Within the function it is passed into `test`; assigned or updated later in the function.

function `sourceLooksC(source = "")`

Purpose and intuition: `sourceLooksC` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means sourceLooksC performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without sourceLooksC, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

text: text stores a computed value for temporary calculation. It is initialized from `String(source || "")`. Within the function it is passed into test; assigned or updated later in the function.

function textBlocksFromPlainText(text)

Purpose and intuition: textBlocksFromPlainText belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: text is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: No other named helper from this file is detected inside the body. That means textBlocksFromPlainText performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without textBlocksFromPlainText, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function unwrapFormula(value)

Purpose and intuition: unwrapFormula belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means unwrapFormula performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without unwrapFormula, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

text: text stores a computed value for temporary calculation. It is initialized from `String(value || "").trim()`. Within the function it is returned to the caller; accesses properties/methods `match`; assigned or updated later in the function.

wrappers: wrappers stores a list for temporary calculation. It is initialized from `[]`. Within the function it is assigned or updated later in the function.

destructured [pattern, group]: destructured [pattern, group] starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It unpacks API handles from another object, giving the rest of the file short direct names for those APIs.

match: match stores a computed value for temporary calculation. It is initialized from `text.match(pattern)`. Within the function it is returned to the caller; assigned or updated later in the function.

function looksLikeBareWebAddress(value = "")

Purpose and intuition: `looksLikeBareWebAddress` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `looksLikeBareWebAddress` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `looksLikeBareWebAddress`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from `String(value || "").trim()`. Within the function it accesses properties/methods `split`; passed into `test`; assigned or updated later in the function.

host: host stores a computed value for temporary calculation. It is initialized from `clean.split(/[/?#]/)[0].toLowerCase()`. Within the function it is passed into `test`; assigned or updated later in the function.

function displayTitle(value, fallback = "Untitled item")

Purpose and intuition: `displayTitle` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `fallback` is the caller-supplied input

used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means displayTitle performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without displayTitle, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

title: title stores a computed value for temporary calculation. It is initialized from `String(value || fallback).trim()`. Within the function it is assigned or updated later in the function.

replacements: replacements stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; assigned or updated later in the function.

function slugLabel(value)

Purpose and intuition: slugLabel belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means slugLabel performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without slugLabel, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

category: category stores a function or callback value for portfolio data state. It is initialized from `categories.find((item) => item.id === value)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is returned to the caller; accesses properties/methods label; assigned or updated later in the function.

function parsedItemTerms(item)

Purpose and intuition: parsedItemTerms interprets dItemTerms text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

Parameters and data it receives: item is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: No other named helper from this file is detected inside the body. That means parsedItemTerms performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without parsedItemTerms, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function pillList(items, className = "")

Purpose and intuition: pillList belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: items is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. className is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means pillList performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without pillList, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

label: label stores a computed value for temporary calculation. It is initialized from typeof item === "string" ? item : item.name || item.title || item.label || "". Within the function it is returned to the caller; assigned or updated later in the function.

function evidenceList(items, renderItem, emptyMessage)

Purpose and intuition: evidenceList belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: items is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. renderItem is the caller-supplied input used by this function. The function interprets it

according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `emptyMessage` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: No other named helper from this file is detected inside the body. That means `evidenceList` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `evidenceList`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function detailBlock(title, className, content)

Purpose and intuition: `detailBlock` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `title` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `className` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `content` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: No other named helper from this file is detected inside the body. That means `detailBlock` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `detailBlock`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function mediaGrid(items)

Purpose and intuition: `mediaGrid` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `items` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `escapeHtml`, `normalizeLinkTarget`, `isWebsiteLinkItem`, `linkAttributes`. Those calls show that `mediaGrid` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `mediaGrid`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function itemLabel(item = {}, fallback = "Download file")

Purpose and intuition: `itemLabel` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `fallback` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `itemUrl`, `fileNameFromUrl`. Those calls show that `itemLabel` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `itemLabel`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

url: `url` stores a computed value for network or routing value. It is initialized from `itemUrl(item)`. Within the function it is passed into `fileNameFromUrl`; assigned or updated later in the function.

function parsedChildCards(children, projectId, sectionIndex, basePath = [])

Purpose and intuition: `parsedChildCards` interprets `dChildCards` text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

Parameters and data it receives: `children` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `projectId` identifies or carries the project being built, rendered, compiled,

searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `sectionIndex` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `basePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `nodeHasRenderableContent`, `nodeOverview`, `parsedNodeCard`. Those calls show that `parsedChildCards` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `parsedChildCards`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

visibleChildren: `visibleChildren` stores a function or callback value for temporary calculation. It is initialized from `children.filter((child) => nodeHasRenderableContent(child) && !child.url && !nodeOverview(child))`. Within the function it accesses properties/methods `length`; assigned or updated later in the function.

function escapeRegExp(value = "")

Purpose and intuition: `escapeRegExp` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `escapeRegExp` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `escapeRegExp`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function scrollToDomTarget(id)

Purpose and intuition: `scrollToDomTarget` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: id is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: Inside the body, it updates browser route/history. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Important body details: updates browser history so Back, Forward, and refresh can follow the same window state.

What it calls or works with: It works with closeSectionDialogForRoute, queueSearchHighlights. Those calls show that scrollToDomTarget is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without scrollToDomTarget, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

target: target stores a DOM element reference. It is initialized from `id ? document.getElementById(id) : null`. Within the function it accesses properties/methods `scrollIntoView`; assigned or updated later in the function.

Profile and site content

The functions in this group all support profile and site content. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function normalizeFunFacts(value)

Purpose and intuition: normalizeFunFacts standardizes FunFacts data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means normalizeFunFacts performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without normalizeFunFacts, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

items: items stores a computed value for temporary calculation. It is initialized from `Array.isArray(value) ? value : String(value || "").split(/\r?\n/)`. Within the function it is returned to the caller; assigned or updated later in the function.

function normalizeSiteContent(content = {})

Purpose and intuition: normalizeSiteContent standardizes SiteContent data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: content is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: No other named helper from this file is detected inside the body. That means normalizeSiteContent performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without normalizeSiteContent, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function normalizeProfile(profileValue = {})

Purpose and intuition: normalizeProfile standardizes Profile data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: profileValue points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: No other named helper from this file is detected inside the body. That means normalizeProfile performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without normalizeProfile, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function mailComposeLink(email = "")

Purpose and intuition: mailComposeLink belongs to profile and site content. This function renders or normalizes owner identity, front-page text, contact links, fun facts, or custom main-page sections. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: email is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means mailComposeLink performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without mailComposeLink, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from `String(email || "").trim()`. Within the function it is returned to the caller; passed into `encodeURIComponent`; assigned or updated later in the function.

function phoneLink(phone = "")

Purpose and intuition: phoneLink belongs to profile and site content. This function renders or normalizes owner identity, front-page text, contact links, fun facts, or custom main-page sections. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: phone is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means phoneLink performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without phoneLink, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from `String(phone || "").trim()`. Within the function it is returned to the caller; accesses properties/methods `replace`; assigned or updated later in the function.

function looksLikeWebOrContactText(value = "")

Purpose and intuition: looksLikeWebOrContactText belongs to profile and site content. This function renders or normalizes owner identity, front-page text, contact links, fun facts, or custom main-page sections. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `looksLikeBareWebAddress`. Those calls show that `looksLikeWebOrContactText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `looksLikeWebOrContactText`, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from `String(value || "").trim()`. Within the function it is returned to the caller; accesses properties/methods `split`; passed into `test`; assigned or updated later in the function.

function isWebsiteLinkItem(item = {}, target = "")

Purpose and intuition: `isWebsiteLinkItem` is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

Parameters and data it receives: item is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. target is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `isWebsiteLinkItem` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `isWebsiteLinkItem`, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from `String(target || "").trim()`. Within the function it is passed into `test`; assigned or updated later in the function.

typeText: typeText stores a list for temporary calculation. It is initialized from `[item.kind, item.type, item.status, item.meta, item.label, item.title]`. Within the function it is passed into `test`; assigned or updated later in the function.

function inferResponsiveProfile(project)

Purpose and intuition: inferResponsiveProfile makes an educated classification from incomplete evidence. It looks at names, extensions, source text, repository state, or runtime state and returns the app's best decision so the next operation can choose the correct path.

Parameters and data it receives: project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: It works with sectionHasRenderableContent, responsiveNodeCount, responsiveNodeHasMedia, hasPublicTemplate. Those calls show that inferResponsiveProfile is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without inferResponsiveProfile, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

sections: sections stores a computed value for portfolio data state. It is initialized from project?.portfolioView?.sections || []. Within the function it is iterated or transformed as a collection; accesses properties/methods filter; assigned or updated later in the function.

visibleSections: visibleSections stores a function or callback value for portfolio data state. It is initialized from sections.filter((section) => section?.id !== "brief" && sectionHasRenderableContent(section)). Within the function it is iterated or transformed as a collection; accesses properties/methods length, reduce, some; assigned or updated later in the function.

itemCount: itemCount stores a computed value for temporary calculation. It is initialized from visibleSections.reduce(). Within the function it is assigned or updated later in the function.

maxDepth: maxDepth stores a computed value for temporary calculation. It is initialized from visibleSections.reduce(). Within the function it is assigned or updated later in the function.

hasMedia: hasMedia stores a function or callback value for temporary calculation. It is initialized from visibleSections.some((section) => responsiveNodeHasMedia(section)). Within the function it is assigned or updated later in the function.

density: density stores a computed value for temporary calculation. It is initialized from itemCount >= 18 || visibleSections.length >= 6 || maxDepth >= 4. Within the function it is assigned or updated later in the function.

function projectResponsiveProfile(project)

Purpose and intuition: projectResponsiveProfile handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Parameters and data it receives: project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: It works with `inferResponsiveProfile`. Those calls show that `projectResponsiveProfile` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `projectResponsiveProfile`, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

savedProfile: `savedProfile` stores a computed value for portfolio data state. It is initialized from `project?.portfolioView?.responsive`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

Rich content rendering

The functions in this group all support rich content rendering. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function renderFunFacts()

Purpose and intuition: `renderFunFacts` turns saved data for FunFacts into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

What it calls or works with: It works with `normalizeFunFacts`, `renderRichContent`, `renderInlineMath`. Those calls show that `renderFunFacts` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `renderFunFacts`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

facts: `facts` stores a computed value for temporary calculation. It is initialized from `normalizeFunFacts(funFacts)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `join`, `length`, `map`; assigned or updated later in the function.

hasRichFacts: `hasRichFacts` stores a computed value for temporary calculation. It is initialized from `Boolean(funFactsRich?.blocks?.length)`. Within the function it is assigned or updated later in the function.

function escapeHtml(value)

Purpose and intuition: escapeHtml belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means escapeHtml performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without escapeHtml, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function normalizeCodeLanguage(value = "")

Purpose and intuition: normalizeCodeLanguage standardizes CodeLanguage data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means normalizeCodeLanguage performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without normalizeCodeLanguage, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from `String(value || "").trim().toLowerCase().replace(/[ _-]+/g, " ")`. Within the function it is passed into includes; assigned or updated later in the function.

match: match stores a function or callback value for temporary calculation. It is initialized from `supportedCodeLanguages.find((language) => ...)`. Within the function it is returned to the caller; assigned or updated later in the function.

function codeLanguageLabel(value = "")

Purpose and intuition: codeLanguageLabel belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with normalizeCodeLanguage. Those calls show that codeLanguageLabel is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without codeLanguageLabel, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function normalizeCodePasteMode(value = "")

Purpose and intuition: normalizeCodePasteMode standardizes CodePasteMode data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means normalizeCodePasteMode performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without normalizeCodePasteMode, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function normalizeCodeText(value = "", pasteMode = "source")

Purpose and intuition: normalizeCodeText standardizes CodeText data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. pasteMode is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with normalizeCodePasteMode. Those calls show that normalizeCodeText is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without normalizeCodeText, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

normalized: normalized stores a computed value for temporary calculation. It is initialized from `String(value || "").replace(/\r\n?/g, "\n").replace(/\u00a0/g, " ")`. Within the function it is returned to the caller; accesses properties/methods `replace`; assigned or updated later in the function.

function detectCodeLanguage(value = "")

Purpose and intuition: detectCodeLanguage makes an educated classification from incomplete evidence. It looks at names, extensions, source text, repository state, or runtime state and returns the app's best decision so the next operation can choose the correct path.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with sourceLooksCpp, sourceLooksC. Those calls show that detectCodeLanguage is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without detectCodeLanguage, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

code: code stores a computed value for temporary calculation. It is initialized from `String(value || "")`. Within the function it is passed into `sourceLooksC`, `sourceLooksCpp`, `test`; assigned or updated later in the function.

function tokenizedCodeHtml(code = "", language = "javascript")

Purpose and intuition: tokenizedCodeHtml is the lightweight syntax highlighter used by the public site. It escapes the code first, then wraps comments, strings, numbers, keywords, preprocessor directives, and other

recognized tokens in spans with CSS classes. It supports the major languages that the builder exposes, including C, C++, SystemVerilog, LTspice-oriented text, Java, JavaScript, Python, and HTML-like code.

Purpose and intuition: The function exists so code added to a project reads like code rather than plain paragraph text. Without it, source examples in the portfolio would be visually flat, harder to scan, and less convincing to technical recruiters.

Parameters and data it receives: code is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. language names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `escapeHtml`, `normalizeCodeLanguage`. Those calls show that `tokenizedCodeHtml` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `tokenizedCodeHtml`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

html: `html` stores a computed value for temporary calculation. It is initialized from `escapeHtml(code)`. Within the function it is returned to the caller; accesses properties/methods `replace`; assigned or updated later in the function.

tokens: `tokens` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `length`, `push`; assigned or updated later in the function.

protect: `protect` stores a function or callback value for temporary calculation. It is initialized from `(pattern, className) => {}`. Within the function it is assigned or updated later in the function.

token: `token` stores a string value for temporary calculation. It is initialized from ``@@@CODE_TOKEN_${tokens.length}@@``. Within the function it is returned to the caller; passed into `protect`; assigned or updated later in the function.

commentPattern: `commentPattern` stores a list for temporary calculation. It is initialized from `["python", "ltspice"].includes(normalizeCodeLanguage(language))`. Within the function it is passed into `protect`; assigned or updated later in the function.

keywordMap: `keywordMap` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is accesses properties/methods `javascript`; assigned or updated later in the function.

keywords: `keywords` stores a computed value for lookup collection. It is initialized from `keywordMap[normalizeCodeLanguage(language)] || keywordMap.javascript`. Within the function it is passed into `replace`; assigned or updated later in the function.

function renderRichCodeBlock(block = {})

Purpose and intuition: renderRichCodeBlock turns saved data for RichCodeBlock into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that styles.css expects.

Parameters and data it receives: block is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with normalizeCodeLanguage, detectCodeLanguage, normalizeCodeText, escapeHtml, codeLanguageLabel, tokenizedCodeHtml. Those calls show that renderRichCodeBlock is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without renderRichCodeBlock, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

language: language stores a computed value for compiler or language configuration. It is initialized from normalizeCodeLanguage(block.language || detectCodeLanguage(block.code || "")). It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into escapeHtml, normalizeCodeLanguage, tokenizedCodeHtml; assigned or updated later in the function.

code: code stores a computed value for temporary calculation. It is initialized from normalizeCodeText(block.code || "", block.pasteMode || "source"). Within the function it is passed into normalizeCodeLanguage, normalizeCodeText, tokenizedCodeHtml; assigned or updated later in the function.

function renderPlainMultiline(value = "")

Purpose and intuition: renderPlainMultiline turns saved data for PlainMultiline into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that styles.css expects.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with escapeHtml. Those calls show that renderPlainMultiline is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without renderPlainMultiline, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function renderSiteContent()

Purpose and intuition: renderSiteContent turns saved data for SiteContent into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that styles.css expects.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

What it calls or works with: It works with normalizeSiteContent, renderRichFieldContent, applyPlainFieldStyle. Those calls show that renderSiteContent is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without renderSiteContent, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

content: content stores a computed value for temporary calculation. It is initialized from normalizeSiteContent(siteContent || {}). Within the function it is accesses properties/methods heroCopy, heroEyebrow, heroTitle; passed into renderRichFieldContent; assigned or updated later in the function.

function renderProfile()

Purpose and intuition: renderProfile applies owner identity to the public page. It updates the displayed name, tagline, profile photo, brand/title text, email link, phone link, GitHub/resume links, contact cards, resume viewer, metadata hints, and social/contact links. It is the bridge between saved profile data and the visible recruiter-facing contact section.

Purpose and intuition: The function is important because the same website shell can serve Maurice's portfolio or a fresh user's portfolio. Profile data cannot be hardcoded if the builder is intended to be reusable. Without renderProfile, the page would either remain generic or require manual HTML editing for every owner.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser DOM. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Important body details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

What it calls or works with: It works with normalizeProfile, mailComposeLink, phoneLink, normalizeLinkTarget, renderRichFieldContent, applyPlainFieldStyle, plainFieldStyleAttribute. Those calls show that renderProfile is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without renderProfile, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

current: current stores a computed value for temporary calculation. It is initialized from normalizeProfile(profile || {}). Within the function it is accesses properties/methods brandImage, brandText, contactIntro, displayName, email, githubUrl, herolImage, linkedinUrl; passed into mailComposeLink, normalizeLinkTarget, phoneLink, renderRichFieldContent; assigned or updated later in the function.

displayName: displayName stores a computed value for temporary calculation. It is initialized from current.displayName || "Portfolio". Within the function it is accesses properties/methods slice; passed into renderRichFieldContent, setAttribute; assigned or updated later in the function.

label: label stores a computed value for temporary calculation. It is initialized from current.portfolioLabel || "Portfolio". Within the function it is passed into applyPlainFieldStyle, renderRichFieldContent, setAttribute; assigned or updated later in the function.

emailLink: emailLink stores a computed value for temporary calculation. It is initialized from mailComposeLink(current.email). Within the function it is assigned or updated later in the function.

callLink: callLink stores a computed value for temporary calculation. It is initialized from phoneLink(current.phone). Within the function it is assigned or updated later in the function.

links: links stores a list for temporary calculation. It is initialized from [. Within the function it is iterated or transformed as a collection; accesses properties/methods length, map; passed into renderRichFieldContent; assigned or updated later in the function.

heroGithubLink: heroGithubLink stores a list for Git publishing and authorization state. It is initialized from [...document.querySelectorAll(".hero-actions a")]. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods hidden, href; assigned or updated later in the function.

resumeObject: resumeObject stores a DOM element reference. It is initialized from resumeSection.querySelector("object"). Within the function it is accesses properties/methods data; assigned or updated later in the function.

fallbackLink: fallbackLink stores a DOM element reference. It is initialized from resumeSection.querySelector("object a"). Within the function it is accesses properties/methods href; assigned or updated later in the function.

styleId: styleId stores a computed value for appearance configuration. It is initialized from link.label === "GitHub". Within the function it is passed into plainFieldStyleAttribute; assigned or updated later in the function.

function renderInlineMath(text)

Purpose and intuition: renderInlineMath turns saved data for InlineMath into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that styles.css expects.

Parameters and data it receives: text is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `escapeHtml`, `normalizeLinkTarget`. Those calls show that `renderInlineMath` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `renderInlineMath`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

escaped: `escaped` stores a computed value for temporary calculation. It is initialized from `escapeHtml(text)`. Within the function it is accesses properties/methods `replace`; assigned or updated later in the function.

withMath: `withMath` stores a computed value for temporary calculation. It is initialized from `escaped.replace(/^$([^\$]+)$/g, '<span class="rich-inline-formula">$1</span>')`. Within the function it is returned to the caller; accesses properties/methods `replace`; assigned or updated later in the function.

trailing: `trailing` stores a computed value for temporary calculation. It is initialized from `match.match(/[/\],...)`. Within the function it is accesses properties/methods `length`; passed into `slice`; assigned or updated later in the function.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `trailing ? match.slice(0, -trailing.length) : match`. Within the function it is passed into `normalizeLinkTarget`; assigned or updated later in the function.

href: `href` stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(clean, { assumeWeb: true })`. Within the function it is assigned or updated later in the function.

function renderMultilineInlineText(text = "")

Purpose and intuition: `renderMultilineInlineText` turns saved data for `MultilineInlineText` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

Parameters and data it receives: `text` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `renderInlineMath`. Those calls show that `renderMultilineInlineText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `renderMultilineInlineText`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function sanitizeRichInlineHtml(value = "")

Purpose and intuition: `sanitizeRichInlineHtml` is the safety filter for rich inline HTML. Builder editors may store spans, links, bold text, colors, and other inline markup, but the public website should not blindly inject arbitrary HTML. This function removes unsafe tags and attributes while preserving the formatting the portfolio actually needs.

Purpose and intuition: The function prevents a content field from becoming a script injection surface. It works with `renderRichContent` and `renderRichFieldContent`, which need formatted HTML but should only receive controlled markup.

Parameters and data it receives: `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Important body details: creates DOM elements instead of relying only on static HTML; writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

What it calls or works with: It works with `normalizeLinkTarget`, `cleanFontFamily`, `normalizeFontPx`, `normalizeTextColor`, `linkifyRichTextNodes`. Those calls show that `sanitizeRichInlineHtml` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `sanitizeRichInlineHtml`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

template: `template` stores a DOM element reference. It is initialized from `document.createElement("template")`. Within the function it is returned to the caller; accesses properties/methods `content`, `innerHTML`; passed into `createElement`, `linkifyRichTextNodes`; assigned or updated later in the function.

allowedTags: `allowedTags` stores a constructed object for lookup collection. It is initialized from `new Set(["A", "B", "BR", "EM", "I", "SPAN", "STRONG", "U"])`. Within the function it is accesses properties/methods `has`; assigned or updated later in the function.

href: `href` stores a computed value for temporary calculation. It is initialized from `node.tagName === "A" ? String(node.getAttribute("href") || "").trim() : ""`. Within the function it is passed into `String`, `normalizeLinkTarget`, `setAttribute`; assigned or updated later in the function.

normalizedHref: `normalizedHref` stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(href, { assumeWeb: true })`. Within the function it is passed into `test`; assigned or updated later in the function.

safeHref: `safeHref` stores a computed value for temporary calculation. It is initialized from `/^(https?:|mailto:|tel:| #|V)/i.test(normalizedHref) ? normalizedHref : ""`. Within the function it is passed into `setAttribute`; assigned or updated later in the function.

fontFamily: `fontFamily` stores a computed value for temporary calculation. It is initialized from `cleanFontFamily(node.style.fontFamily || "")`. Within the function it is passed into `cleanFontFamily`; assigned or updated later in the function.

fontPx: fontPx stores a computed value for temporary calculation. It is initialized from `normalizeFontPx(parseFloat(node.style.fontSize || ""))`. Within the function it is assigned or updated later in the function.

color: color stores a computed value for appearance configuration. It is initialized from `normalizeTextColor(node.style.color || "")`. Within the function it is passed into `normalizeTextColor`; assigned or updated later in the function.

rawFontWeight: rawFontWeight stores a computed value for temporary calculation. It is initialized from `node.style.fontWeight || ""`. Within the function it is passed into `test`; assigned or updated later in the function.

fontWeight: fontWeight stores a computed value for temporary calculation. It is initialized from `/(bold|[6-9]00)/i.test(rawFontWeight)`. Within the function it is assigned or updated later in the function.

fontStyle: fontStyle stores a computed value for appearance configuration. It is initialized from `node.style.fontStyle === "italic" ? "italic" : ""`. Within the function it is assigned or updated later in the function.

textDecoration: textDecoration stores a computed value for temporary calculation. It is initialized from `node.style.textDecoration.includes("underline") ? "underline" : ""`. Within the function it accesses properties/methods `includes`; assigned or updated later in the function.

function richTextStyle(block = {})

Purpose and intuition: richTextStyle belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: block is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `cleanFontFamily`, `normalizeFontPx`, `normalizeTextColor`, `escapeHtml`. Those calls show that richTextStyle is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without richTextStyle, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

styles: styles stores a list for appearance configuration. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `join`, `length`, `push`; passed into `escapeHtml`; assigned or updated later in the function.

fontFamily: fontFamily stores a computed value for temporary calculation. It is initialized from `cleanFontFamily(block.fontFamily || "Arial") || "Arial"`. Within the function it is passed into `cleanFontFamily`, `push`; assigned or updated later in the function.

fontPx: fontPx stores a computed value for temporary calculation. It is initialized from `normalizeFontPx(block.fontPx)`. Within the function it is passed into `normalizeFontPx`, push; assigned or updated later in the function.

color: color stores a computed value for appearance configuration. It is initialized from `normalizeTextColor(block.color || "")`. Within the function it is passed into `normalizeTextColor`, push; assigned or updated later in the function.

function richImageCropStyle(block = {})

Purpose and intuition: `richImageCropStyle` belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `block` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `normalizeCropAspect`, `normalizeCropZoom`, `normalizeCropPosition`, `escapeHtml`. Those calls show that `richImageCropStyle` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `richImageCropStyle`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

aspect: `aspect` stores a computed value for temporary calculation. It is initialized from `normalizeCropAspect(block.cropAspect)`. Within the function it is passed into push; assigned or updated later in the function.

styles: `styles` stores a list for appearance configuration. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `push`; passed into `escapeHtml`; assigned or updated later in the function.

function richImageDownloadLink(block = {})

Purpose and intuition: `richImageDownloadLink` belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `block` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `escapeHtml`, `cleanRichImageTitle`, `normalizeLinkTarget`, `isWebsiteLinkItem`, `linkAttributes`, `downloadAttribute`. Those calls show that `richImageDownloadLink` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `richImageDownloadLink`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

label: label stores a computed value for temporary calculation. It is initialized from `escapeHtml(cleanRichImageTitle(block) || block.caption || "Image file")`. Within the function it is assigned or updated later in the function.

url: url stores a computed value for network or routing value. It is initialized from `block.url || "#"`. Within the function it is passed into `normalizeLinkTarget`; assigned or updated later in the function.

target: target stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(url, { assumeWeb: isWebsiteLinkItem(block, url) })`. Within the function it is passed into `downloadAttribute`, `escapeHtml`, `linkAttributes`; assigned or updated later in the function.

function cleanRichImageTitle(block = {})

Purpose and intuition: `cleanRichImageTitle` belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `block` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `cleanRichImageTitle` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `cleanRichImageTitle`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

title: title stores a computed value for temporary calculation. It is initialized from `String(block.title || "").trim()`. Within the function it is passed into `String`, `test`; assigned or updated later in the function.

function renderRichContent(rich, fallbackText = "")

Purpose and intuition: `renderRichContent` is the renderer for builder-created rich content. It accepts blocks representing paragraphs, images, formulas, files, links, code blocks, captions, and styled text, then converts those blocks into safe public HTML. It decides whether an image should appear inline, whether a file should be downloadable, how captions attach, and how text styles survive into the website.

Purpose and intuition: This function matters because the builder is not just saving plain text. It supports formatted explanations, pasted images, formulas, code, and files. Without `renderRichContent`, much of what the builder collects would be flattened or lost when the portfolio is published.

Parameters and data it receives: `rich` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `fallbackText` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `textBlocksFromPlainText`, `richImageDownloadLink`, `cleanRichImageTitle`, `normalizeLinkTarget`, `normalizeCropAspect`, `richImageCropStyle`, `escapeHtml`, `unwrapFormula`. Those calls show that `renderRichContent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `renderRichContent`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

blocks: `blocks` stores a computed value for temporary calculation. It is initialized from `rich?.blocks?.length ? rich.blocks : textBlocksFromPlainText(fallbackText)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `map`; assigned or updated later in the function.

align: `align` stores a list for temporary calculation. It is initialized from `["left", "center", "right"].includes(block.align) ? block.align : "left"`. Within the function it is passed into `includes`; assigned or updated later in the function.

title: `title` stores a computed value for temporary calculation. It is initialized from `cleanRichImageTitle(block)`. Within the function it is passed into `escapeHtml`; assigned or updated later in the function.

imageSrc: `imageSrc` stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(block.url, { assumeWeb: true })`. Within the function it is passed into `escapeHtml`; assigned or updated later in the function.

formula: `formula` stores a computed value for temporary calculation. It is initialized from `unwrapFormula(block.formula)`. Within the function it is passed into `escapeHtml`, `renderInlineMath`, `unwrapFormula`; assigned or updated later in the function.

size: `size` stores a list for temporary calculation. It is initialized from `["small", "normal", "large"].includes(block.fontSize) ? block.fontSize : "normal"`. Within the function it is assigned or updated later in the function.

content: `content` stores a computed value for temporary calculation. It is initialized from `block.html`. Within the function it is assigned or updated later in the function.

function `renderRichFieldContent(rich, fallbackText = "")`

Purpose and intuition: `renderRichFieldContent` turns saved data for `RichFieldContent` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

Parameters and data it receives: rich is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. fallbackText is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with textBlocksFromPlainText, sanitizeRichInlineHtml, renderInlineMath, richTextStyle. Those calls show that renderRichFieldContent is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without renderRichFieldContent, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

blocks: blocks stores a computed value for temporary calculation. It is initialized from rich?.blocks?.length ? rich.blocks : textBlocksFromPlainText(fallbackText). Within the function it is returned to the caller; iterated or transformed as a collection; accesses properties/methods map; assigned or updated later in the function.

align: align stores a list for temporary calculation. It is initialized from ["left", "center", "right"].includes(block.align) ? block.align : "left". Within the function it is passed into includes; assigned or updated later in the function.

content: content stores a computed value for temporary calculation. It is initialized from block.html. Within the function it is assigned or updated later in the function.

function richTextTerms(rich)

Purpose and intuition: richTextTerms belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: rich is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means richTextTerms performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without richTextTerms, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function renderDynamicLinks(items = [])

Purpose and intuition: renderDynamicLinks turns saved data for DynamicLinks into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that styles.css expects.

Parameters and data it receives: items is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with resourceLink. Those calls show that renderDynamicLinks is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without renderDynamicLinks, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

links: links stores a function or callback value for temporary calculation. It is initialized from items.filter((item) => item?.url && (item.label || item.title)). Within the function it is iterated or transformed as a collection; accesses properties/methods length, map; assigned or updated later in the function.

function collectRichDownloads(rich, fallbackType = "File", output = [])

Purpose and intuition: collectRichDownloads gathers scattered nested data into a shape that another function can loop over. This is important because portfolio data is hierarchical while rendering, search, and AI context often need a flat list.

Parameters and data it receives: rich is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. fallbackType is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. output is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with cleanRichImageTitle, fileNameFromUrl. Those calls show that collectRichDownloads is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without collectRichDownloads, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function renderDownloadBlock(title, items = [])

Purpose and intuition: renderDownloadBlock turns saved data for DownloadBlock into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that styles.css expects.

Parameters and data it receives: title is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. items is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with uniqueDownloads, detailBlock, escapeHtml, resourceLink, renderMultilineInlineText. Those calls show that renderDownloadBlock is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without renderDownloadBlock, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

downloads: downloads stores a computed value for temporary calculation. It is initialized from uniqueDownloads(items). Within the function it is iterated or transformed as a collection; accesses properties/methods length, map; assigned or updated later in the function.

function renderParsedBriefBlock(section, fallbackSummary = "")

Purpose and intuition: renderParsedBriefBlock turns saved data for ParsedBriefBlock into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that styles.css expects.

Parameters and data it receives: section is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. fallbackSummary is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with renderRichContent. Those calls show that renderParsedBriefBlock is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `renderParsedBriefBlock`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

briefItem: `briefItem` stores a computed value for temporary calculation. It is initialized from `section?.items?.[0] || {}`. Within the function it is accesses properties/methods `description`, `rich`; passed into `renderRichContent`; assigned or updated later in the function.

briefText: `briefText` stores a computed value for temporary calculation. It is initialized from `briefItem.description || fallbackSummary || ""`. Within the function it is passed into `renderRichContent`; assigned or updated later in the function.

function richHasRenderableContent(rich)

Purpose and intuition: `richHasRenderableContent` belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `rich` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `richHasRenderableContent` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `richHasRenderableContent`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function richContainsMedia(rich)

Purpose and intuition: `richContainsMedia` belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `rich` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `richContainsMedia` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without richContainsMedia, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function renderProjects()

Purpose and intuition: renderProjects rebuilds the visible project directory from current state. It checks the active category filter and search query, groups visible projects by category, creates the category sections, and updates the summary count. It is called after catalog load, filter changes, and search changes.

Purpose and intuition: The function is the public directory refresh mechanism. Without it, filters and searches might update state internally but the page would not visually change.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Important body details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

What it calls or works with: It works with normalize, projectVisible, queueSearchHighlights, categorySection. Those calls show that renderProjects is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without renderProjects, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

query: query stores a computed value for temporary calculation. It is initialized from `normalize(searchInput.value).trim()`. Within the function it is passed into `projectVisible`; assigned or updated later in the function.

visible: visible stores a function or callback value for temporary calculation. It is initialized from `projects.filter((project) => projectVisible(project, query))`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `length`; assigned or updated later in the function.

visibleProjects: visibleProjects stores a function or callback value for portfolio data state. It is initialized from `visible.filter((project) => project.category === category.id)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is returned to the caller; accesses properties/methods `length`; passed into `categorySection`; assigned or updated later in the function.

shouldShowEmptyCategory: shouldShowEmptyCategory stores a computed value for portfolio data state. It is initialized from `!query && (activeFilter === "all" || activeFilter === category.id)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

Project windows and navigation

The functions in this group all support project windows and navigation. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function clampSectionDialogPosition(left, top, dialog)

Purpose and intuition: clampSectionDialogPosition belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: left is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. top is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. dialog is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: No other named helper from this file is detected inside the body. That means clampSectionDialogPosition performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without clampSectionDialogPosition, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

rect: rect stores a computed value for temporary calculation. It is initialized from dialog.getBoundingClientRect(). Within the function it is accesses properties/methods height, width; passed into max; assigned or updated later in the function.

margin: margin stores a numeric value for temporary calculation. It is initialized from 12. Within the function it is passed into max, min; assigned or updated later in the function.

maxLeft: maxLeft stores a computed value for temporary calculation. It is initialized from Math.max(margin, window.innerWidth - rect.width - margin). Within the function it is assigned or updated later in the function.

maxTop: maxTop stores a computed value for temporary calculation. It is initialized from Math.max(margin, window.innerHeight - rect.height - margin). Within the function it is assigned or updated later in the function.

function anchorSectionDialog(dialog)

Purpose and intuition: anchorSectionDialog belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: dialog is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: toggles CSS classes to express UI state.

What it calls or works with: It works with `clampSectionDialogPosition`. Those calls show that `anchorSectionDialog` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `anchorSectionDialog`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

rect: `rect` stores a computed value for temporary calculation. It is initialized from `dialog.getBoundingClientRect()`. Within the function it is accesses properties/methods `left`, `top`; passed into `clampSectionDialogPosition`; assigned or updated later in the function.

position: `position` stores a computed value for temporary calculation. It is initialized from `clampSectionDialogPosition(rect.left, rect.top, dialog)`. Within the function it is accesses properties/methods `left`, `top`; assigned or updated later in the function.

function ensureSectionResizeHandles(dialog)

Purpose and intuition: `ensureSectionResizeHandles` creates or repairs something only when it is needed. This pattern avoids duplicate DOM nodes, duplicate repositories, duplicate handlers, or missing folders while still letting later code assume the resource exists.

Parameters and data it receives: `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: creates DOM elements instead of relying only on static HTML.

What it calls or works with: No other named helper from this file is detected inside the body. That means `ensureSectionResizeHandles` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `ensureSectionResizeHandles`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

handle: `handle` stores a DOM element reference. It is initialized from `document.createElement("div")`. Within the function it is accesses properties/methods `className`, `dataset`, `setAttribute`; passed into `append`; assigned or updated later in the function.

function anchorSectionDialogForResize(dialog)

Purpose and intuition: `anchorSectionDialogForResize` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: toggles CSS classes to express UI state.

What it calls or works with: It works with `anchorSectionDialog`. Those calls show that `anchorSectionDialogForResize` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `anchorSectionDialogForResize`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

rect: `rect` stores a computed value for temporary calculation. It is initialized from `dialog.getBoundingClientRect()`. Within the function it is accesses properties/methods `height`, `width`; assigned or updated later in the function.

function sectionDialogResizeBounds(state, event)

Purpose and intuition: `sectionDialogResizeBounds` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `state` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `event` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: No other named helper from this file is detected inside the body. That means `sectionDialogResizeBounds` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `sectionDialogResizeBounds`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

margin: `margin` stores a numeric value for temporary calculation. It is initialized from 12. Within the function it is passed into `min`; assigned or updated later in the function.

minWidth: `minWidth` stores a computed value for temporary calculation. It is initialized from `Math.min(320, Math.max(180, window.innerWidth - margin * 2))`. Within the function it is passed into `min`; assigned or updated later in the function.

minHeight: `minHeight` stores a computed value for temporary calculation. It is initialized from `Math.min(170, Math.max(120, window.innerHeight - margin * 2))`. Within the function it is passed into `min`; assigned or updated later in the function.

direction: direction stores a computed value for filesystem location. It is initialized from state.direction. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods includes; assigned or updated later in the function.

destructured { left, top, width, height }: destructured { left, top, width, height } stores a computed value for imported or unpacked API handles. It is initialized from state.rect. It unpacks API handles from another object, giving the rest of the file short direct names for those APIs.

dx: dx stores a computed value for temporary calculation. It is initialized from event.clientX - state.startX. Within the function it is passed into min; assigned or updated later in the function.

dy: dy stores a computed value for temporary calculation. It is initialized from event.clientY - state.startY. Within the function it is passed into min; assigned or updated later in the function.

right: right stores a computed value for temporary calculation. It is initialized from state.rect.left + state.rect.width. Within the function it is assigned or updated later in the function.

bottom: bottom stores a computed value for temporary calculation. It is initialized from state.rect.top + state.rect.height. Within the function it is assigned or updated later in the function.

function sectionDialogFromDragEvent(event)

Purpose and intuition: sectionDialogFromDragEvent belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: event is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means sectionDialogFromDragEvent performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without sectionDialogFromDragEvent, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

handle: handle stores a computed value for temporary calculation. It is initialized from event.target.closest("[data-section-dialog-drag='true']"). Within the function it is accesses properties/methods closest; assigned or updated later in the function.

dialog: dialog stores a computed value for DOM reference or UI state. It is initialized from handle.closest("dialog"). It anchors UI behavior to a specific browser element or window state. Within the function it is returned to the caller; passed into closest; assigned or updated later in the function.

function beginSectionDialogDrag(event)

Purpose and intuition: beginSectionDialogDrag belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: event is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: toggles CSS classes to express UI state.

What it calls or works with: It works with sectionDialogFromDragEvent, anchorSectionDialog. Those calls show that beginSectionDialogDrag is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without beginSectionDialogDrag, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

dialog: dialog stores a computed value for DOM reference or UI state. It is initialized from sectionDialogFromDragEvent(event). It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods classList, getBoundingClientRect; passed into add, anchorSectionDialog; assigned or updated later in the function.

rect: rect stores a computed value for temporary calculation. It is initialized from dialog.getBoundingClientRect(). Within the function it is accesses properties/methods left, top; assigned or updated later in the function.

function moveSectionDialogDrag(event)

Purpose and intuition: moveSectionDialogDrag belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: event is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: It works with clampSectionDialogPosition. Those calls show that moveSectionDialogDrag is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without moveSectionDialogDrag, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

next: next stores a computed value for temporary calculation. It is initialized from `clampSectionDialogPosition()`. Within the function it is accesses properties/methods `left`, `top`; assigned or updated later in the function.

function endSectionDialogDrag()

Purpose and intuition: `endSectionDialogDrag` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: toggles CSS classes to express UI state.

What it calls or works with: No other named helper from this file is detected inside the body. That means `endSectionDialogDrag` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `endSectionDialogDrag`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function beginSectionDialogResize(event)

Purpose and intuition: `beginSectionDialogResize` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `event` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: toggles CSS classes to express UI state.

What it calls or works with: It works with `anchorSectionDialogForResize`. Those calls show that `beginSectionDialogResize` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `beginSectionDialogResize`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

handle: handle stores a computed value for temporary calculation. It is initialized from `event.target.closest("[data-section-dialog-resize-handle]")`. Within the function it is accesses properties/methods dataset; passed into `closest`; assigned or updated later in the function.

dialog: dialog stores a computed value for DOM reference or UI state. It is initialized from `handle?.closest("dialog")`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods `classList`, `getBoundingClientRect`; passed into `add`, `anchorSectionDialogForResize`, `closest`; assigned or updated later in the function.

function moveSectionDialogResize(event)

Purpose and intuition: `moveSectionDialogResize` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `event` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: It works with `sectionDialogResizeBounds`. Those calls show that `moveSectionDialogResize` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `moveSectionDialogResize`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

next: next stores a computed value for temporary calculation. It is initialized from `sectionDialogResizeBounds(activeSectionDialogResize, event)`. Within the function it is accesses properties/methods `height`, `left`, `top`, `width`; assigned or updated later in the function.

function endSectionDialogResize()

Purpose and intuition: `endSectionDialogResize` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: toggles CSS classes to express UI state.

What it calls or works with: No other named helper from this file is detected inside the body. That means `endSectionDialogResize` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `endSectionDialogResize`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function `updateSectionDialogMinimize(dialog)`

Purpose and intuition: `updateSectionDialogMinimize` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: toggles CSS classes to express UI state.

What it calls or works with: No other named helper from this file is detected inside the body. That means `updateSectionDialogMinimize` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `updateSectionDialogMinimize`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

minimized: `minimized` stores a computed value for temporary calculation. It is initialized from `dialog.classList.contains("is-minimized-dialog")`. Within the function it is passed into `contains`, `setAttribute`; assigned or updated later in the function.

button: `button` stores a DOM element reference. It is initialized from `dialog.querySelector(".section-view-minimize")`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods `setAttribute`, `textContent`, `title`; assigned or updated later in the function.

function `toggleSectionDialogMinimized(dialog)`

Purpose and intuition: `toggleSectionDialogMinimized` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: toggles CSS classes to express UI state.

What it calls or works with: It works with `anchorSectionDialog`, `updateSectionDialogMinimize`. Those calls show that `toggleSectionDialogMinimized` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `toggleSectionDialogMinimized`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function enableSectionDialogDrag()

Purpose and intuition: `enableSectionDialogDrag` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: attaches event handlers that turn user actions into behavior.

What it calls or works with: It works with `clampSectionDialogPosition`. Those calls show that `enableSectionDialogDrag` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `enableSectionDialogDrag`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

rect: `rect` stores a computed value for temporary calculation. It is initialized from `dialog.getBoundingClientRect()`. Within the function it is accesses properties/methods `left`, `top`; passed into `clampSectionDialogPosition`; assigned or updated later in the function.

next: `next` stores a computed value for temporary calculation. It is initialized from `clampSectionDialogPosition(rect.left, rect.top, dialog)`. Within the function it is accesses properties/methods `left`, `top`; assigned or updated later in the function.

function linkifyRichTextNodes(root)

Purpose and intuition: `linkifyRichTextNodes` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `root` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Important body details: creates DOM elements instead of relying only on static HTML.

What it calls or works with: It works with `normalizeLinkTarget`. Those calls show that `linkifyRichTextNodes` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `linkifyRichTextNodes`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

walker: walker stores a DOM element reference. It is initialized from `document.createTreeWalker(root, NodeFilter.SHOW_TEXT, {`. Within the function it is accesses properties/methods `nextNode`; assigned or updated later in the function.

nodes: nodes stores a list for temporary calculation. It is initialized from `[]`. Within the function it is iterated or transformed as a collection; mutated as a collection or DOM container; accesses properties/methods `forEach`, `push`; assigned or updated later in the function.

node: node stores a computed value for temporary calculation. It is initialized from `walker.nextNode()`. Within the function it is returned to the caller; accesses properties/methods `parentElement`, `textContent`; passed into `createTreeWalker`, `push`, `test`; assigned or updated later in the function.

text: text stores a computed value for temporary calculation. It is initialized from `textNode.textContent || ""`. Within the function it is accesses properties/methods `length`, `replace`, `slice`; passed into `append`; assigned or updated later in the function.

fragment: fragment stores a DOM element reference. It is initialized from `document.createDocumentFragment()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `append`; passed into `replaceWith`; assigned or updated later in the function.

cursor: cursor stores a numeric value for temporary calculation. It is initialized from 0. Within the function it is passed into `append`; assigned or updated later in the function.

trailing: trailing stores a computed value for temporary calculation. It is initialized from `match.match(/I),..`. Within the function it is accesses properties/methods `length`; passed into `append`, `slice`; assigned or updated later in the function.

clean: clean stores a computed value for temporary calculation. It is initialized from `trailing ? match.slice(0, -trailing.length) : match`. Within the function it is passed into `normalizeLinkTarget`; assigned or updated later in the function.

link: link stores a DOM element reference. It is initialized from `document.createElement("a")`. Within the function it is accesses properties/methods `href`, `rel`, `target`, `textContent`; passed into `append`; assigned or updated later in the function.

function siteSectionHasContent(section)

Purpose and intuition: `siteSectionHasContent` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `section` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `siteSectionHasContent` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `siteSectionHasContent`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function siteSectionRenderable(section)

Purpose and intuition: `siteSectionRenderable` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `section` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `siteSectionHasContent`. Those calls show that `siteSectionRenderable` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `siteSectionRenderable`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function renderSiteSections()

Purpose and intuition: `renderSiteSections` turns saved data for `SiteSections` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser DOM. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Important body details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

What it calls or works with: It works with `escapeHtml`, `renderRichContent`, `renderDynamicLinks`. Those calls show that `renderSiteSections` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `renderSiteSections`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

mount: `mount` stores a DOM element reference. It is initialized from `document.querySelector("#dynamic-sections")`. Within the function it is accesses properties/methods `innerHTML`; assigned or updated later in the function.

visibleSections: `visibleSections` stores a function or callback value for portfolio data state. It is initialized from `(siteSections || []).filter(siteSectionRenderable)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `map`; assigned or updated later in the function.

style: `style` stores a computed value for appearance configuration. It is initialized from `section.backgroundImage ? `style="--section-bg: url('${escapeHtml(section.backgroundImage)}')'` : ""`. Within the function it is assigned or updated later in the function.

links: `links` stores a list for temporary calculation. It is initialized from `[...(section.links || []), ...(section.assets || [])]`. Within the function it is passed into `renderDynamicLinks`; assigned or updated later in the function.

subsections: `subsections` stores a function or callback value for portfolio data state. It is initialized from `(section.subsections || []).filter((item) => item.title || item.description || item.richDescription?.blocks?.length || (item.links || []).length)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; assigned or updated later in the function.

function customSectionBlocks(project)

Purpose and intuition: `customSectionBlocks` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `detailBlock`, `evidenceList`, `resourceLink`. Those calls show that `customSectionBlocks` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `customSectionBlocks`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function renderCollectedDownloadSections(project)

Purpose and intuition: renderCollectedDownloadSections turns saved data for CollectedDownloadSections into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that styles.css expects.

Parameters and data it receives: project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with uniqueDownloads, collectDownloadsFromValue, renderDownloadBlock, displayTitle. Those calls show that renderCollectedDownloadSections is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without renderCollectedDownloadSections, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

blocks: blocks stores a list for temporary calculation. It is initialized from []. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods join, push; assigned or updated later in the function.

addBlock: addBlock stores a function or callback value for temporary calculation. It is initialized from (title, source, fallbackType = "File") => {}. Within the function it is assigned or updated later in the function.

downloads: downloads stores a computed value for temporary calculation. It is initialized from uniqueDownloads(collectDownloadsFromValue(source, fallbackType, [])). Within the function it is accesses properties/methods length; passed into addBlock, push; assigned or updated later in the function.

function pathToString(path = [])

Purpose and intuition: pathToString belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: path points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Important body details: uses Node path utilities so Windows path separators and relative paths are handled consistently.

What it calls or works with: No other named helper from this file is detected inside the body. That means pathToString performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without pathToString, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function pathFromString(value = "")

Purpose and intuition: pathFromString belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: No other named helper from this file is detected inside the body. That means pathFromString performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without pathFromString, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function sectionRouteState(projectId, sectionIndex, resourcePath = "")

Purpose and intuition: sectionRouteState belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: projectId identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. sectionIndex is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. resourcePath points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: No other named helper from this file is detected inside the body. That means sectionRouteState performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `sectionRouteState`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function `canUseSectionHistory()`

Purpose and intuition: `canUseSectionHistory` is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser route/history. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Important body details: updates browser history so Back, Forward, and refresh can follow the same window state.

What it calls or works with: No other named helper from this file is detected inside the body. That means `canUseSectionHistory` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `canUseSectionHistory`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function `sectionBaseUrl()`

Purpose and intuition: `sectionBaseUrl` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `sectionBaseUrl` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `sectionBaseUrl`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

url: url stores a constructed object for network or routing value. It is initialized from new URL(window.location.href). Within the function it is returned to the caller; accesses properties/methods searchParams; assigned or updated later in the function.

function sectionRouteUrl(projectId, sectionIndex, resourcePath = "")

Purpose and intuition: sectionRouteUrl belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: projectId identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. sectionIndex is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. resourcePath points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with sectionBaseUrl. Those calls show that sectionRouteUrl is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without sectionRouteUrl, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

url: url stores a computed value for network or routing value. It is initialized from sectionBaseUrl(). Within the function it is returned to the caller; accesses properties/methods hash, searchParams; assigned or updated later in the function.

function sectionRouteFromState(state)

Purpose and intuition: sectionRouteFromState belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: state is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: No other named helper from this file is detected inside the body. That means sectionRouteFromState performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `sectionRouteFromState`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function `sectionRouteFromLocation()`

Purpose and intuition: `sectionRouteFromLocation` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser route/history. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Important body details: reads or writes URL query parameters so browser history can represent the current view.

What it calls or works with: No other named helper from this file is detected inside the body. That means `sectionRouteFromLocation` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `sectionRouteFromLocation`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

params: `params` stores a constructed object for temporary calculation. It is initialized from `new URLSearchParams(window.location.search)`. Within the function it is accesses properties/methods `get`; assigned or updated later in the function.

projectId: `projectId` stores a computed value for portfolio data state. It is initialized from `params.get(sectionRouteParams.project)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

sectionIndex: `sectionIndex` stores a computed value for portfolio data state. It is initialized from `params.get(sectionRouteParams.section)`. Within the function it is assigned or updated later in the function.

function `sameSectionRoute(left, right)`

Purpose and intuition: `sameSectionRoute` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `left` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `right` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `sameSectionRoute` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `sameSectionRoute`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function syncSectionRouteToHistory(projectId, sectionIndex, resourcePath = "", mode = "push")

Purpose and intuition: `syncSectionRouteToHistory` reconciles two states that can drift apart. In this project, sync functions connect local drafts, route state, Git branches, publish mirrors, or frontend state to the current truth.

Parameters and data it receives: `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `sectionIndex` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `resourcePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `mode` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it updates browser route/history. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: It works with `canUseSectionHistory`, `sectionRouteState`, `sectionRouteUrl`, `sectionRouteFromState`, `sectionRouteFromLocation`, `sameSectionRoute`. Those calls show that `syncSectionRouteToHistory` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `syncSectionRouteToHistory`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

route: `route` stores a resolved filesystem or route value. It is initialized from `sectionRouteState(projectId, sectionIndex, resourcePath)`. Within the function it is passed into `sameSectionRoute`; assigned or updated later in the function.

url: `url` stores a resolved filesystem or route value. It is initialized from `sectionRouteUrl(projectId, sectionIndex, resourcePath)`. Within the function it is assigned or updated later in the function.

currentRoute: currentRoute stores a computed value for temporary calculation. It is initialized from `sectionRouteFromState(history.state) || sectionRouteFromLocation()`. Within the function it is passed into `sameSectionRoute`; assigned or updated later in the function.

method: method stores a computed value for temporary calculation. It is initialized from `mode === "replace" || sameSectionRoute(currentRoute, route) ? "replaceState" : "pushState"`. Within the function it is assigned or updated later in the function.

function clearSectionRouteInHistory()

Purpose and intuition: `clearSectionRouteInHistory` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser route/history. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: updates browser history so Back, Forward, and refresh can follow the same window state.

What it calls or works with: It works with `canUseSectionHistory`, `sectionBaseUrl`. Those calls show that `clearSectionRouteInHistory` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `clearSectionRouteInHistory`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function initializeSectionHistoryState()

Purpose and intuition: `initializeSectionHistoryState` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser route/history. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: updates browser history so Back, Forward, and refresh can follow the same window state.

What it calls or works with: It works with `canUseSectionHistory`, `sectionRouteFromLocation`, `sectionRouteFromState`, `sectionRouteState`, `sectionRouteUrl`. Those calls show that `initializeSectionHistoryState` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `initializeSectionHistoryState`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

route: route stores a computed value for temporary calculation. It is initialized from `sectionRouteFromLocation() || sectionRouteFromState(history.state)`. Within the function it is accesses properties/methods `projectId`, `resourcePath`, `sectionIndex`; passed into `replaceState`, `sectionRouteUrl`; assigned or updated later in the function.

function nodeAtPath(section, path = [])

Purpose and intuition: `nodeAtPath` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `section` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `path` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `nodeAtPath` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `nodeAtPath`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

node: node stores a computed value for temporary calculation. It is initialized from `section`. Within the function it is returned to the caller; accesses properties/methods `children`; assigned or updated later in the function.

children: `children` stores a computed value for temporary calculation. It is initialized from `section?.items || []`. Within the function it is assigned or updated later in the function.

index: `index` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function.

function nodeChildren(node)

Purpose and intuition: `nodeChildren` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `nodeChildren` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `nodeChildren`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function nodeHasRenderableContent(node)

Purpose and intuition: `nodeHasRenderableContent` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `richHasRenderableContent`, `nodeChildren`. Those calls show that `nodeHasRenderableContent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `nodeHasRenderableContent`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

children: `children` stores a computed value for temporary calculation. It is initialized from `nodeChildren(node)`. Within the function it accesses properties/methods `some`; assigned or updated later in the function.

function sectionHasRenderableContent(section)

Purpose and intuition: `sectionHasRenderableContent` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `section` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `richHasRenderableContent`. Those calls show that `sectionHasRenderableContent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `sectionHasRenderableContent`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function responsiveNodeHasMedia(node)

Purpose and intuition: `responsiveNodeHasMedia` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `richContainsMedia`, `responsiveChildren`. Those calls show that `responsiveNodeHasMedia` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `responsiveNodeHasMedia`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

url: `url` stores a computed value for network or routing value. It is initialized from `String(node.url || "")`. Within the function it is passed into `String`, `test`; assigned or updated later in the function.

function responsiveNodeCount(node)

Purpose and intuition: `responsiveNodeCount` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `nodeHasRenderableContent`, `responsiveChildren`. Those calls show that `responsiveNodeCount` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `responsiveNodeCount`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function responsiveNodeDepth(node)

Purpose and intuition: `responsiveNodeDepth` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `nodeHasRenderableContent`, `responsiveChildren`. Those calls show that `responsiveNodeDepth` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `responsiveNodeDepth`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

children: `children` stores a computed value for temporary calculation. It is initialized from `responsiveChildren(node).filter(nodeHasRenderableContent)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; passed into `max`; assigned or updated later in the function.

function nodeSummary(title, rich, text, emptyMessage = "No summary has been added yet.")

Purpose and intuition: `nodeSummary` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `title` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `rich` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `text` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `emptyMessage` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `escapeHtml`, `renderRichContent`. Those calls show that `nodeSummary` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `nodeSummary`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function `nodeSummary(item = {})`

Purpose and intuition: `nodeSummary` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `normalize`, `displayTitle`. Those calls show that `nodeSummary` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `nodeSummary`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

title: `title` stores a computed value for temporary calculation. It is initialized from `normalize(displayTitle(item.title || item.label || ""))`. Within the function it is accessed properties/methods `endsWith`; passed into `normalize`; assigned or updated later in the function.

function `nodeOverviewDetails(node, children = [])`

Purpose and intuition: `nodeOverviewDetails` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. `children` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `renderRichContent`, `renderInlineSectionFiles`, `nodeChildren`. Those calls show that `nodeOverviewDetails` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `nodeOverviewDetails`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

overviewItems: `overviewItems` stores a computed value for temporary calculation. It is initialized from `children.filter(nodesOverview)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

blocks: `blocks` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `length`, `push`; assigned or updated later in the function.

itemFiles: `itemFiles` stores a computed value for temporary calculation. It is initialized from `renderInlineSectionFiles(nodeChildren(item))`. Within the function it is passed into `push`; assigned or updated later in the function.

function `parsedNodeCard(node, projectId, sectionIndex, path)`

Purpose and intuition: `parsedNodeCard` interprets `dNodeCard` text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

Parameters and data it receives: `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `sectionIndex` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `path` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `displayTitle`, `pathToString`, `escapeHtml`. Those calls show that `parsedNodeCard` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `parsedNodeCard`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

title: `title` stores a computed value for temporary calculation. It is initialized from `displayTitle(node.title)`. Within the function it is passed into `displayTitle`, `escapeHtml`; assigned or updated later in the function.

function parsedNodeContent(node, projectId, sectionIndex, path = [])

Purpose and intuition: parsedNodeContent interprets dNodeContent text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

Parameters and data it receives: node is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. projectId identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. sectionIndex is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. path points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with renderInlineSectionFiles, nodeChildren, nodeOverview, nodeOverviewDetails, parsedChildCards. Those calls show that parsedNodeContent is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without parsedNodeContent, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

isContainer: isContainer stores a computed value for temporary calculation. It is initialized from !node.kind || node.kind === "subsection". Within the function it is assigned or updated later in the function.

children: children stores a computed value for temporary calculation. It is initialized from nodeChildren(node). Within the function it is iterated or transformed as a collection; accesses properties/methods filter; passed into nodeOverviewDetails, parsedChildCards; assigned or updated later in the function.

contentChildren: contentChildren stores a function or callback value for temporary calculation. It is initialized from children.filter((child) => !nodeOverview(child)). Within the function it is passed into renderInlineSectionFiles; assigned or updated later in the function.

function parsedSectionContent(section, projectId, sectionIndex, path = [])

Purpose and intuition: parsedSectionContent interprets dSectionContent text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

Parameters and data it receives: section is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. projectId identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. sectionIndex is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. path points at an external resource, local file, route, Git remote, or public link. The function

normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `nodeAtPath`, `parsedNodeContent`. Those calls show that `parsedSectionContent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `parsedSectionContent`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

node: node stores a resolved filesystem or route value. It is initialized from `path.length ? nodeAtPath(section, path) : section`. Within the function it is passed into `parsedNodeContent`; assigned or updated later in the function.

function `parsedSection(section, index, project)`

Purpose and intuition: `parsedSection` interprets `dSection` text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

Parameters and data it receives: `section` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `index` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `escapeHtml`, `displayTitle`. Those calls show that `parsedSection` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `parsedSection`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

visibleItems: `visibleItems` stores a function or callback value for temporary calculation. It is initialized from `(section.items || []).filter(nodeHasRenderableContent)`. Within the function it is accesses properties/methods some; assigned or updated later in the function.

hasImages: hasImages stores a function or callback value for temporary calculation. It is initialized from `visibleItems.some((item) => item.kind === "image")`. Within the function it is assigned or updated later in the function.

function categorySection(category, visibleProjects)

Purpose and intuition: categorySection handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Parameters and data it receives: category is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. visibleProjects identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with escapeHtml. Those calls show that categorySection is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without categorySection, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function ensureSectionDialog()

Purpose and intuition: ensureSectionDialog creates and wires the reusable full-window dialog used for project sections. It builds the top controls, back/forward behavior, close behavior, content container, and event delegation for child cards. It creates the window once and then later calls reuse it.

Purpose and intuition: The function exists so every project section window behaves consistently. If each click created its own markup by hand, close buttons, back buttons, keyboard behavior, and nested navigation would drift apart.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Important body details: serializes structured data before sending or saving it; creates DOM elements instead of relying only on static HTML; writes generated markup into the page, which is why escaping and sanitizing helpers around it matter; toggles CSS classes to express UI state; attaches event handlers that turn user actions into behavior.

What it calls or works with: It works with closeSectionDialog, clearSectionRouteInHistory, updateSectionDialogMinimize, pathFromString, sectionForwardStack, pathToString, openParsedSection. Those calls show that ensureSectionDialog is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `ensureSectionDialog`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

dialog: `dialog` stores a DOM element reference. It is initialized from `document.querySelector("#section-view-dialog")`. It anchors UI behavior to a specific browser element or window state. Within the function it is returned to the caller; accesses properties/methods `addEventListener`, `classList`, `className`, `dataset`, `id`, `innerHTML`, `querySelector`; passed into `append`, `closeSectionDialog`, `createElement`, `openParsedSection`, `pathFromString`, `querySelector`, `remove`; assigned or updated later in the function.

returnTarget: `returnTarget` stores a DOM element reference. It is initialized from `document.querySelector()`. Within the function it is assigned or updated later in the function.

path: `path` stores a resolved filesystem or route value. It is initialized from `pathFromString(dialog.dataset.resourcePath || "")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accessed properties/methods `length`, `pop`; passed into `closest`, `openParsedSection`, `push`; assigned or updated later in the function.

stack: `stack` stores a computed value for runtime state or cache. It is initialized from `sectionForwardStack(dialog)`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `pop`, `push`; passed into `stringify`; assigned or updated later in the function.

stack: `stack` stores a computed value for runtime state or cache. It is initialized from `sectionForwardStack(dialog)`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `pop`, `push`; passed into `stringify`; assigned or updated later in the function.

nextPath: `nextPath` stores a computed value for filesystem location. It is initialized from `stack.pop()`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `openParsedSection`; assigned or updated later in the function.

nodeButton: `nodeButton` stores a computed value for DOM reference or UI state. It is initialized from `event.target.closest("[data-resource-path]")`. It anchors UI behavior to a specific browser element or window state. Within the function it is accessed properties/methods `dataset`; passed into `openParsedSection`; assigned or updated later in the function.

function sectionForwardStack(dialog)

Purpose and intuition: `sectionForwardStack` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Important body details: parses JSON rather than treating incoming text as already trusted data.

What it calls or works with: No other named helper from this file is detected inside the body. That means `sectionForwardStack` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `sectionForwardStack`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function `updateSectionNavigation(dialog, path)`

Purpose and intuition: `updateSectionNavigation` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. `path` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: It works with `sectionForwardStack`. Those calls show that `updateSectionNavigation` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `updateSectionNavigation`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

back: `back` stores a DOM element reference. It is initialized from `dialog.querySelector(".section-view-back")`. Within the function it is accesses properties/methods `hidden`, `setAttribute`, `title`; passed into `querySelector`; assigned or updated later in the function.

function `closeSectionDialog(dialog)`

Purpose and intuition: `closeSectionDialog` reverses an open window or route state. It does more than hide markup: it must also clean navigation state, restore the previous view when appropriate, and avoid trapping the visitor inside a dialog.

Parameters and data it receives: `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: It works with `clearSectionRouteInHistory`. Those calls show that `closeSectionDialog` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `closeSectionDialog`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function `closeOrStepBackSectionDialog(dialog)`

Purpose and intuition: `closeOrStepBackSectionDialog` reverses an open window or route state. It does more than hide markup: it must also clean navigation state, restore the previous view when appropriate, and avoid trapping the visitor inside a dialog.

Parameters and data it receives: `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: serializes structured data before sending or saving it.

What it calls or works with: It works with `pathFromString`, `sectionForwardStack`, `pathToString`, `openParsedSection`, `closeSectionDialog`. Those calls show that `closeOrStepBackSectionDialog` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `closeOrStepBackSectionDialog`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

path: `path` stores a resolved filesystem or route value. It is initialized from `pathFromString(dialog.dataset.resourcePath || "")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it accesses properties/methods `length`, `pop`; passed into `openParsedSection`, `push`; assigned or updated later in the function.

stack: `stack` stores a computed value for runtime state or cache. It is initialized from `sectionForwardStack(dialog)`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `push`; passed into `stringify`; assigned or updated later in the function.

function `openParsedSection(projectId, sectionIndex, resourcePath = "", options = {})`

Purpose and intuition: `openParsedSection` opens a project section or nested subsection as a full-window public view. It finds the project, finds the section, walks the requested subsection path, applies the selected appearance template, renders the overview and child sections, updates navigation buttons, and syncs browser history.

Purpose and intuition: This function is the heart of the website's project-window behavior. Without it, clicking Design, Simulation, Documentation, or any user-created section would either scroll the page awkwardly or render content in the wrong place.

Parameters and data it receives: `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `sectionIndex` is the caller-supplied input used by this function. The function interprets it

according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `resourcePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Important body details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter; toggles CSS classes to express UI state.

What it calls or works with: It works with `sectionHasRenderableContent`, `pathFromString`, `nodeAtPath`, `nodeHasRenderableContent`, `ensureSectionDialog`, `applyProjectTemplateToElement`, `pathToString`, `syncSectionRouteToHistory`. Those calls show that `openParsedSection` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `openParsedSection`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

project: `project` stores a function or callback value for portfolio data state. It is initialized from `projects.find((item) => item.id === projectId)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `category`; passed into `applyProjectTemplateToElement`; assigned or updated later in the function.

sections: `sections` stores a function or callback value for portfolio data state. It is initialized from `(project?.portfolioView?.sections || []).filter((section) => section.id !== "brief" && sectionHasRenderableContent(section))`. Within the function it is assigned or updated later in the function.

section: `section` stores a computed value for portfolio data state. It is initialized from `sections[Number(sectionIndex)]`. Within the function it is accesses properties/methods `id`, `title`; passed into `displayTitle`, `filter`, `nodeAtPath`, `parsedSectionContent`, `querySelector`, `sectionHasRenderableContent`; assigned or updated later in the function.

path: `path` stores a resolved filesystem or route value. It is initialized from `pathFromString(resourcePath)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods `length`; passed into `nodeAtPath`, `pathToString`, `syncSectionRouteToHistory`, `updateSectionNavigation`; assigned or updated later in the function.

node: `node` stores a resolved filesystem or route value. It is initialized from `path.length ? nodeAtPath(section, path) : section`. Within the function it is accesses properties/methods `title`; passed into `displayTitle`; assigned or updated later in the function.

dialog: `dialog` stores a computed value for DOM reference or UI state. It is initialized from `ensureSectionDialog()`. It anchors UI behavior to a specific browser element or window state. Within the

function it is accesses properties/methods dataset, open, querySelector, scrollTop, showModal; passed into applyProjectTemplateToElement, updateSectionNavigation; assigned or updated later in the function.

category: category stores a function or callback value for portfolio data state. It is initialized from `categories.find((item) => item.id === project.category) || {}`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods accent; passed into applyProjectTemplateToElement; assigned or updated later in the function.

function closeSectionDialogForRoute()

Purpose and intuition: closeSectionDialogForRoute reverses an open window or route state. It does more than hide markup: it must also clean navigation state, restore the previous view when appropriate, and avoid trapping the visitor inside a dialog.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: No other named helper from this file is detected inside the body. That means closeSectionDialogForRoute performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without closeSectionDialogForRoute, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

dialog: dialog stores a DOM element reference. It is initialized from `document.querySelector("#section-view-dialog")`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods close; passed into querySelector; assigned or updated later in the function.

function applySectionRoute(route)

Purpose and intuition: applySectionRoute belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: route is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: It works with closeSectionDialogForRoute, openParsedSection. Those calls show that applySectionRoute is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without applySectionRoute, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

opened: opened stores a resolved filesystem or route value. It is initialized from `openParsedSection(route.projectId, route.sectionIndex, route.resourcePath, {`. Within the function it is assigned or updated later in the function.

function restoreSectionRouteAfterCatalogLoad()

Purpose and intuition: `restoreSectionRouteAfterCatalogLoad` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser route/history. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: attaches event handlers that turn user actions into behavior; updates browser history so Back, Forward, and refresh can follow the same window state.

What it calls or works with: It works with `initializeSectionHistoryState`, `applySectionRoute`, `sectionRouteFromLocation`, `sectionRouteFromState`, `ensureSearchPanel`, `goToSearchResult`, `clearSearchHighlights`, `closeSectionDialogForRoute`. Those calls show that `restoreSectionRouteAfterCatalogLoad` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `restoreSectionRouteAfterCatalogLoad`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

topLink: `topLink` stores a computed value for temporary calculation. It is initialized from `event.target.closest('a[href="#top"]')`. Within the function it is assigned or updated later in the function.

question: `question` stores a computed value for temporary calculation. It is initialized from `aiAssistantInput?.value || ""`. Within the function it is passed into `answerAssistantQuestion`; assigned or updated later in the function.

button: `button` stores a computed value for DOM reference or UI state. It is initialized from `event.target.closest("[data-ai-source-id]")`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods dataset; passed into `get`; assigned or updated later in the function.

source: `source` stores a computed value for lookup collection. It is initialized from `assistantSourceMap.get(button.dataset.aiSourceId)`. Within the function it is passed into `closest`, `goToSearchResult`; assigned or updated later in the function.

sectionButton: `sectionButton` stores a computed value for portfolio data state. It is initialized from `event.target.closest("[data-section-project]")`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods dataset; passed into `openParsedSection`; assigned or updated later in the function.

function renderInlineSectionFiles(items = [])

Purpose and intuition: renderInlineSectionFiles turns saved data for InlineSectionFiles into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that styles.css expects.

Parameters and data it receives: items is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with uniqueDownloads, normalizeDownloadAsset, escapeHtml, normalizeLinkTarget, isWebsiteLinkItem, linkAttributes, downloadAttribute, fileNameFromUrl. Those calls show that renderInlineSectionFiles is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without renderInlineSectionFiles, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

downloads: downloads stores a function or callback value for lookup collection. It is initialized from uniqueDownloads(items.map((item) => normalizeDownloadAsset(item, "File")).filter(Boolean)). Within the function it is iterated or transformed as a collection; accesses properties/methods length, map; assigned or updated later in the function.

Resources and links

The functions in this group all support resources and links. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function normalizeLinkTarget(target = "", options = {})

Purpose and intuition: normalizeLinkTarget standardizes LinkTarget data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: target is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. options is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `looksLikeBareWebAddress`. Those calls show that `normalizeLinkTarget` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `normalizeLinkTarget`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(target || "").trim()`. Within the function it is returned to the caller; assigned or updated later in the function.

function linkAttributes(url, item = {})

Purpose and intuition: `linkAttributes` belongs to resources and links. This function decides whether content is a website link, local download, uploaded file, or resource card. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `normalizeLinkTarget`, `isWebsiteLinkItem`. Those calls show that `linkAttributes` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `linkAttributes`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

target: `target` stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(url, { assumeWeb: isWebsiteLinkItem(item, url) })`. Within the function it is passed into `test`; assigned or updated later in the function.

function isLocalDownloadTarget(target = "")

Purpose and intuition: `isLocalDownloadTarget` is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

Parameters and data it receives: `target` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `normalizeLinkTarget`. Those calls show that `isLocalDownloadTarget` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `isLocalDownloadTarget`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

value: value stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(target, { assumeWeb: true })`. Within the function it is passed into `Boolean`, `test`; assigned or updated later in the function.

function downloadAttribute(target = "", item = {})

Purpose and intuition: `downloadAttribute` belongs to resources and links. This function decides whether content is a website link, local download, uploaded file, or resource card. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `target` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `normalizeLinkTarget`, `isWebsiteLinkItem`, `isLocalDownloadTarget`. Those calls show that `downloadAttribute` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `downloadAttribute`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

value: value stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(target, { assumeWeb: isWebsiteLinkItem(item, target) })`. Within the function it is passed into `isLocalDownloadTarget`; assigned or updated later in the function.

```
function fileNameFromUrl(url = "")
```

Purpose and intuition: fileNameFromUrl belongs to resources and links. This function decides whether content is a website link, local download, uploaded file, or resource card. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: url points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means fileNameFromUrl performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without fileNameFromUrl, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from `String(url || "").split(/[?#]/)[0].split("/").pop() || "Download file"`. Within the function it is returned to the caller; passed into `decodeURIComponent`; assigned or updated later in the function.

```
function resourceLink(item = {}, label = item.label || item.title || item.name || fileNameFromUrl(item.url || item.artifact))
```

Purpose and intuition: resourceLink belongs to resources and links. This function decides whether content is a website link, local download, uploaded file, or resource card. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: item is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. label is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `normalizeLinkTarget`, `isWebsiteLinkItem`, `escapeHtml`, `linkAttributes`, `downloadAttribute`, `fileNameFromUrl`. Those calls show that resourceLink is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without resourceLink, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

rawTarget: rawTarget stores a computed value for temporary calculation. It is initialized from `item.url || item.artifact || item.href || item.file || item.path || item.src || ""`. Within the function it is passed into `normalizeLinkTarget`; assigned or updated later in the function.

target: target stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(rawTarget, { assumeWeb: isWebsiteLinkItem(item, rawTarget) })`. Within the function it is passed into `downloadAttribute`, `escapeHtml`, `linkAttributes`; assigned or updated later in the function.

function itemUrl(item = {})

Purpose and intuition: itemUrl belongs to resources and links. This function decides whether content is a website link, local download, uploaded file, or resource card. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: item is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means itemUrl performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without itemUrl, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function normalizeDownloadAsset(item = {}, fallbackType = "File")

Purpose and intuition: normalizeDownloadAsset standardizes DownloadAsset data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: item is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. fallbackType is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: It works with `itemUrl`, `itemLabel`. Those calls show that `normalizeDownloadAsset` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `normalizeDownloadAsset`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

url: url stores a computed value for network or routing value. It is initialized from `itemUrl(item)`. Within the function it is assigned or updated later in the function.

function collectDownloadsFromValue(value, fallbackType = "File", output = [])

Purpose and intuition: `collectDownloadsFromValue` gathers scattered nested data into a shape that another function can loop over. This is important because portfolio data is hierarchical while rendering, search, and AI context often need a flat list.

Parameters and data it receives: `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `fallbackType` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `output` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `normalizeDownloadAsset`, `collectRichDownloads`. Those calls show that `collectDownloadsFromValue` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `collectDownloadsFromValue`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

asset: asset stores a computed value for lookup collection. It is initialized from `normalizeDownloadAsset(value, fallbackType)`. Within the function it is passed into `push`; assigned or updated later in the function.

function uniqueDownloads(items = [])

Purpose and intuition: `uniqueDownloads` belongs to resources and links. This function decides whether content is a website link, local download, uploaded file, or resource card. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `items` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or

rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `uniqueDownloads` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `uniqueDownloads`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

seen: `seen` stores a constructed object for lookup collection. It is initialized from `new Set()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `add`, `has`; assigned or updated later in the function.

key: `key` stores a string value for temporary calculation. It is initialized from ``${item.url}:${item.title}``. Within the function it is passed into `add`; assigned or updated later in the function.

Appearance and responsive presentation

The functions in this group all support appearance and responsive presentation. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function `cleanFontFamily(value = "")`

Purpose and intuition: `cleanFontFamily` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `cleanFontFamily` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `cleanFontFamily`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function normalizeFontPx(value)

Purpose and intuition: normalizeFontPx standardizes FontPx data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means normalizeFontPx performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without normalizeFontPx, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

size: size stores a computed value for temporary calculation. It is initialized from Number(value). Within the function it is passed into String, isFinite; assigned or updated later in the function.

function normalizeTextColor(value = "")

Purpose and intuition: normalizeTextColor standardizes TextColor data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means normalizeTextColor performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without normalizeTextColor, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

color: color stores a computed value for appearance configuration. It is initialized from String(value || "").trim(). Within the function it is passed into test; assigned or updated later in the function.

function rgbColorToHex(value = "")

Purpose and intuition: rgbColorToHex belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: No other named helper from this file is detected inside the body. That means `rgbColorToHex` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `rgbColorToHex`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

color: color stores a computed value for appearance configuration. It is initialized from `String(value || "").trim()`. Within the function it is returned to the caller; accesses properties/methods `match`, `toLowerCase`; assigned or updated later in the function.

shortHex: shortHex stores a computed value for temporary calculation. It is initialized from `color.match(/^#([0-9a-f])([0-9a-f])([0-9a-f])$/i)`. Within the function it is assigned or updated later in the function.

rgb: rgb stores a computed value for temporary calculation. It is initialized from `color.match(/^rgba?\s*((\d+)\s*(\d+)\s*(\d+)\s*(\d+)?)/i)`. Within the function it is assigned or updated later in the function.

function `normalizePlainFieldStyle(style = {})`

Purpose and intuition: `normalizePlainFieldStyle` standardizes `PlainFieldStyle` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: style is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `cleanFontFamily`, `normalizeFontPx`, `normalizeTextColor`, `rgbColorToHex`. Those calls show that `normalizePlainFieldStyle` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `normalizePlainFieldStyle`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

normalized: normalized stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; accesses properties/methods `bold`, `color`, `fontFamily`, `fontPx`, `italic`, `underline`; assigned or updated later in the function.

fontFamily: `fontFamily` stores a computed value for temporary calculation. It is initialized from `cleanFontFamily(style.fontFamily || "")`. Within the function it is passed into `cleanFontFamily`; assigned or updated later in the function.

fontPx: `fontPx` stores a computed value for temporary calculation. It is initialized from `normalizeFontPx(style.fontPx || style.fontSize || "")`. Within the function it is passed into `normalizeFontPx`; assigned or updated later in the function.

color: `color` stores a computed value for appearance configuration. It is initialized from `normalizeTextColor(style.color || "")`. Within the function it is passed into `normalizeTextColor`, `rgbColorToHex`; assigned or updated later in the function.

function normalizeFieldStyles(styles = {})

Purpose and intuition: `normalizeFieldStyles` standardizes `FieldStyles` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: `styles` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `normalizePlainFieldStyle`. Those calls show that `normalizeFieldStyles` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `normalizeFieldStyles`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function plainFieldStyleToCss(style = {})

Purpose and intuition: `plainFieldStyleToCss` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `style` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `normalizePlainFieldStyle`. Those calls show that `plainFieldStyleToCss` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `plainFieldStyleToCss`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

normalized: `normalized` stores a computed value for temporary calculation. It is initialized from `normalizePlainFieldStyle(style)`. Within the function it is accesses properties/methods `bold`, `color`, `fontFamily`, `fontPx`, `italic`, `underline`; passed into `push`; assigned or updated later in the function.

rules: `rules` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `join`, `push`; assigned or updated later in the function.

function `applyPlainFieldStyle(element, fieldId = "")`

Purpose and intuition: `applyPlainFieldStyle` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `element` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. `fieldId` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: It works with `normalizePlainFieldStyle`. Those calls show that `applyPlainFieldStyle` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `applyPlainFieldStyle`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

style: `style` stores a computed value for appearance configuration. It is initialized from `normalizePlainFieldStyle(fieldStyles[fieldId] || {})`. Within the function it is accesses properties/methods `bold`, `color`, `fontFamily`, `fontPx`, `fontSize`, `fontStyle`, `fontWeight`, `italic`; assigned or updated later in the function.

function plainFieldStyleAttribute(fieldId = "")

Purpose and intuition: plainFieldStyleAttribute belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: fieldId is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with plainFieldStyleToCss, escapeHtml. Those calls show that plainFieldStyleAttribute is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without plainFieldStyleAttribute, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

css: css stores a computed value for temporary calculation. It is initialized from plainFieldStyleToCss(fieldStyles[fieldId] || {}). Within the function it is returned to the caller; passed into escapeHtml; assigned or updated later in the function.

function normalizeCropAspect(value = "")

Purpose and intuition: normalizeCropAspect standardizes CropAspect data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: No other named helper from this file is detected inside the body. That means normalizeCropAspect performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without normalizeCropAspect, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function normalizeCropZoom(value = 1)

Purpose and intuition: normalizeCropZoom standardizes CropZoom data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means normalizeCropZoom performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without normalizeCropZoom, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

zoom: zoom stores a computed value for temporary calculation. It is initialized from Number(value). Within the function it is passed into isFinite, min; assigned or updated later in the function.

function normalizeCropPosition(value = 50)

Purpose and intuition: normalizeCropPosition standardizes CropPosition data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means normalizeCropPosition performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without normalizeCropPosition, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

position: position stores a computed value for temporary calculation. It is initialized from Number(value). Within the function it is passed into isFinite, min; assigned or updated later in the function.

function canonicalTemplateld(id)

Purpose and intuition: canonicalTemplateld is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

Parameters and data it receives: id is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means canonicalTemplateld performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without canonicalTemplateld, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function projectTemplateld(project)

Purpose and intuition: projectTemplateld handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Parameters and data it receives: project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with canonicalTemplateld. Those calls show that projectTemplateld is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without projectTemplateld, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function projectTemplateClass(project)

Purpose and intuition: projectTemplateClass handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Parameters and data it receives: project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `projectTemplateId`. Those calls show that `projectTemplateClass` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `projectTemplateClass`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

templateId: `templateId` stores a computed value for appearance configuration. It is initialized from `projectTemplateId(project)`. Within the function it is returned to the caller; assigned or updated later in the function.

function responsiveClassName(value, fallback)

Purpose and intuition: `responsiveClassName` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `fallback` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `responsiveClassName` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `responsiveClassName`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function projectResponsiveClass(project)

Purpose and intuition: `projectResponsiveClass` handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Parameters and data it receives: `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `projectResponsiveProfile`, `responsiveClassName`. Those calls show that `projectResponsiveClass` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `projectResponsiveClass`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

profile: profile stores a computed value for portfolio data state. It is initialized from `projectResponsiveProfile(project)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `cardLayout`, `density`; passed into `responsiveClassName`; assigned or updated later in the function.

function responsiveStyleValues(project)

Purpose and intuition: `responsiveStyleValues` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: It works with `projectResponsiveProfile`. Those calls show that `responsiveStyleValues` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `responsiveStyleValues`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

profile: profile stores a computed value for portfolio data state. It is initialized from `projectResponsiveProfile(project)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `contentMax`, `mediaMax`, `sectionCardMin`, `touchTarget`; assigned or updated later in the function.

function hasPublicTemplate(project)

Purpose and intuition: `hasPublicTemplate` is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

Parameters and data it receives: `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `projectTemplateId`. Those calls show that `hasPublicTemplate` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `hasPublicTemplate`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function projectTemplateVisual(project)

Purpose and intuition: `projectTemplateVisual` handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Parameters and data it receives: `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `projectTemplateVisual` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `projectTemplateVisual`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function projectTemplateStyle(project, accent)

Purpose and intuition: `projectTemplateStyle` handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Parameters and data it receives: `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `accent` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `projectTemplateVisual`, `responsiveStyleValues`. Those calls show that `projectTemplateStyle` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without projectTemplateStyle, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

visual: visual stores a computed value for temporary calculation. It is initialized from projectTemplateVisual(project) || {}. Within the function it is accesses properties/methods accent, background, click, clickText, hover, line, panel, text; assigned or updated later in the function.

values: values stores a structured object for temporary calculation. It is initialized from {}. Within the function it is passed into entries; assigned or updated later in the function.

function applyProjectTemplateToElement(element, project, accent)

Purpose and intuition: applyProjectTemplateToElement belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: element is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. accent is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: toggles CSS classes to express UI state.

What it calls or works with: It works with projectTemplateClass, projectResponsiveClass, projectTemplateVisual, responsiveStyleValues. Those calls show that applyProjectTemplateToElement is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without applyProjectTemplateToElement, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

visual: visual stores a computed value for temporary calculation. It is initialized from projectTemplateVisual(project) || {}. Within the function it is accesses properties/methods accent, background, click, clickText, hover, line, panel, text; assigned or updated later in the function.

values: values stores a structured object for temporary calculation. It is initialized from {}. Within the function it is passed into entries; assigned or updated later in the function.

function responsiveChildren(node)

Purpose and intuition: responsiveChildren belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: node is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means responsiveChildren performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without responsiveChildren, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function normalizeTextColor(value = "")

Purpose and intuition: normalizeTextColor standardizes TextColor data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means normalizeTextColor performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without normalizeTextColor, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

color: color stores a computed value for appearance configuration. It is initialized from String(value || "").trim(). Within the function it is passed into test; assigned or updated later in the function.

Project directory

The functions in this group all support project directory. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function flattenProject(project)

Purpose and intuition: flattenProject gathers scattered nested data into a shape that another function can loop over. This is important because portfolio data is hierarchical while rendering, search, and AI context often need a flat list.

Parameters and data it receives: project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: It works with slugLabel. Those calls show that flattenProject is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without flattenProject, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

categoryLabel: categoryLabel stores a computed value for portfolio data state. It is initialized from slugLabel(project.category). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into electronicsSearchKeywords; assigned or updated later in the function.

electronicsKeywords: electronicsKeywords stores a computed value for temporary calculation. It is initialized from window.electronicsSearchKeywords. Within the function it is assigned or updated later in the function.

richSummaryTerms: richSummaryTerms stores a function or callback value for lookup collection. It is initialized from (project.summaryRich?.blocks || []).flatMap((block) => [. Within the function it is assigned or updated later in the function.

parsedChildTerms: parsedChildTerms stores a function or callback value for lookup collection. It is initialized from (project.portfolioView?.sections || []).flatMap((section) =>. Within the function it is assigned or updated later in the function.

design: design stores a computed value for temporary calculation. It is initialized from project.design || {}. Within the function it is accesses properties/methods brief, documentation, simulation; assigned or updated later in the function.

designTerms: designTerms stores a list for temporary calculation. It is initialized from [. Within the function it is assigned or updated later in the function.

function projectVisible(project, query)

Purpose and intuition: projectVisible handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Parameters and data it receives: project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. query is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `projectMatches`. Those calls show that `projectVisible` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `projectVisible`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

categoryMatch: `categoryMatch` stores a computed value for portfolio data state. It is initialized from `activeFilter === "all" || project.category === activeFilter`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is returned to the caller; assigned or updated later in the function.

function normalizeCategory(category = {})

Purpose and intuition: `normalizeCategory` standardizes Category data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Parameters and data it receives: `category` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: It works with `normalizeTextColor`. Those calls show that `normalizeCategory` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `normalizeCategory`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

label: `label` stores a computed value for temporary calculation. It is initialized from `String(category.label || category.title || category.id || "Project category").trim() || "Project category"`. Within the function it accesses properties/methods `toLowerCase`; passed into `String`; assigned or updated later in the function.

function hydrateProjectCategories(rawCategories = [], rawProjects = [])

Purpose and intuition: `hydrateProjectCategories` belongs to project directory. This function normalizes, flattens, filters, or renders projects and categories. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `rawCategories` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `rawProjects` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting

work spill across unrelated projects. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `normalizeCategory`. Those calls show that `hydrateProjectCategories` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `hydrateProjectCategories`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

normalized: `normalized` stores a computed value for lookup collection. It is initialized from `rawCategories.map(normalizeCategory)`. Within the function it is returned to the caller; iterated or transformed as a collection; mutated as a collection or DOM container; accesses properties/methods `map`, `push`; passed into `Set`.

knownIds: `knownIds` stores a constructed object for lookup collection. It is initialized from `new Set(normalized.map((category) => category.id))`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `add`, `has`; assigned or updated later in the function.

id: `id` stores a computed value for temporary calculation. It is initialized from `String(project?.category || "").trim()`. Within the function it is passed into `add`, `push`; assigned or updated later in the function.

label: `label` stores a computed value for temporary calculation. It is initialized from `id`. Within the function it is passed into `push`; assigned or updated later in the function.

function projectCard(project)

Purpose and intuition: `projectCard` handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Parameters and data it receives: `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `sectionHasRenderableContent`, `projectTemplateClass`, `projectResponsiveClass`, `projectTemplateStyle`, `renderParsedBriefBlock`, `parsedSection`, `renderMultilineInlineText`, `detailBlock`. Those calls show that `projectCard` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `projectCard`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

category: category stores a function or callback value for portfolio data state. It is initialized from `categories.find((item) => item.id === project.category) || {}`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `accent`; assigned or updated later in the function.

accent: accent stores a computed value for temporary calculation. It is initialized from `category.accent || "#117c7a"`. Within the function it is passed into `projectTemplateStyle`; assigned or updated later in the function.

sections: sections stores a computed value for portfolio data state. It is initialized from `project.portfolioView.sections || []`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `find`; assigned or updated later in the function.

briefSection: briefSection stores a function or callback value for portfolio data state. It is initialized from `sections.find((section) => section.id === "brief")`. Within the function it is passed into `renderParsedBriefBlock`; assigned or updated later in the function.

otherSections: otherSections stores a function or callback value for portfolio data state. It is initialized from `sections.filter((section) => section.id !== "brief" && sectionHasRenderableContent(section))`. Within the function it is iterated or transformed as a collection; accesses properties/methods `map`; assigned or updated later in the function.

category: category stores a function or callback value for portfolio data state. It is initialized from `categories.find((item) => item.id === project.category) || {}`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `accent`; assigned or updated later in the function.

accent: accent stores a computed value for temporary calculation. It is initialized from `category.accent || "#117c7a"`. Within the function it is passed into `projectTemplateStyle`; assigned or updated later in the function.

function loadProjectCatalog()

Purpose and intuition: `loadProjectCatalog` is the public website startup pipeline. It chooses embedded preview data when the builder is previewing, otherwise it fetches `projects.json` from the published site. After that it hydrates the major runtime state: categories, projects, profile, site content, fun facts, custom sections, text styles, and rich text variants. Then it renders the visible page and restores any deep-linked project window.

Purpose and intuition: This function is async because fetching `projects.json` can take time and can fail. It is the reason the static HTML shell becomes a real portfolio. Without it, the page would show placeholders but no saved projects, profile details, resume, fun facts, project categories, search index, or assistant context.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it calls a network resource and updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Important body details: throws explicit errors when a required safety or compatibility condition fails; writes generated markup into the page, which is why escaping and sanitizing helpers around it matter; performs a fetch and therefore must handle network delay and failure.

What it calls or works with: It works with `hydrateProjectCategories`, `normalizeFieldStyles`, `normalizeSiteContent`, `normalizeProfile`, `normalizeFunFacts`, `renderProfile`, `renderSiteContent`, `renderFunFacts`. Those calls show that `loadProjectCatalog` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `loadProjectCatalog`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

catalogUrl: `catalogUrl` stores a constructed object for network or routing value. It is initialized from `new URL("projects.json", window.location.href)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `href`, `searchParams`; passed into `fetch`; assigned or updated later in the function.

Search and filtering

The functions in this group all support search and filtering. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function projectMatches(project, query)

Purpose and intuition: `projectMatches` handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Parameters and data it receives: `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `query` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `flattenProject`. Those calls show that `projectMatches` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `projectMatches`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function setActiveFilter(filter = "all")

Purpose and intuition: `setActiveFilter` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `filter` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: toggles CSS classes to express UI state.

What it calls or works with: No other named helper from this file is detected inside the body. That means setActiveFilter performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without setActiveFilter, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function bindFilterButtons()

Purpose and intuition: bindFilterButtons belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: attaches event handlers that turn user actions into behavior.

What it calls or works with: It works with setActiveFilter, renderProjects, updateSearchDropdown. Those calls show that bindFilterButtons is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without bindFilterButtons, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function renderCategoryFilters()

Purpose and intuition: renderCategoryFilters turns saved data for CategoryFilters into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that styles.css expects.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

What it calls or works with: It works with `escapeHtml`, `bindFilterButtons`, `setActiveFilter`. Those calls show that `renderCategoryFilters` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `renderCategoryFilters`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function `flattenSearchText(values = [])`

Purpose and intuition: `flattenSearchText` gathers scattered nested data into a shape that another function can loop over. This is important because portfolio data is hierarchical while rendering, search, and AI context often need a flat list.

Parameters and data it receives: `values` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Important body details: serializes structured data before sending or saving it.

What it calls or works with: No other named helper from this file is detected inside the body. That means `flattenSearchText` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `flattenSearchText`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function `addSearchEntry(entries, entry)`

Purpose and intuition: `addSearchEntry` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `entries` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `entry` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `flattenSearchText`, `normalize`. Those calls show that `addSearchEntry` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `addSearchEntry`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

text: `text` stores a computed value for temporary calculation. It is initialized from `flattenSearchText([entry.title, entry.type, entry.context, entry.text, entry.url])`. Within the function it is passed into `flattenSearchText`, `normalize`; assigned or updated later in the function.

storedEntry: `storedEntry` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; passed into `push`; assigned or updated later in the function.

function searchSnippet(text = "", query = "")

Purpose and intuition: `searchSnippet` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `text` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `query` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `normalize`. Those calls show that `searchSnippet` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `searchSnippet`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(text || "").replace(/s+/g, " ").trim()`. Within the function it is returned to the caller; accesses properties/methods `length`, `slice`; passed into `min`, `normalize`; assigned or updated later in the function.

index: `index` stores a computed value for temporary calculation. It is initialized from `normalize(clean).indexOf(normalize(query))`. Within the function it is passed into `max`, `min`; assigned or updated later in the function.

start: `start` stores a computed value for temporary calculation. It is initialized from `Math.max(0, index - 54)`. Within the function it is passed into `slice`; assigned or updated later in the function.

end: `end` stores a computed value for temporary calculation. It is initialized from `Math.min(clean.length, index + query.length + 80)`. Within the function it is passed into `slice`; assigned or updated later in the function.

function entryScore(entry, query)

Purpose and intuition: entryScore belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: entry is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. query is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with normalize. Those calls show that entryScore is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without entryScore, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

normalizedQuery: normalizedQuery stores a computed value for temporary calculation. It is initialized from `normalize(query).trim()`. Within the function it is accesses properties/methods `split`; assigned or updated later in the function.

title: title stores a computed value for temporary calculation. It is initialized from `normalize(entry.title)`. Within the function it is accesses properties/methods `includes`, `startsWith`; passed into `normalize`; assigned or updated later in the function.

type: type stores a computed value for temporary calculation. It is initialized from `normalize(entry.type)`. Within the function it is accesses properties/methods `includes`; passed into `normalize`; assigned or updated later in the function.

context: context stores a computed value for temporary calculation. It is initialized from `normalize(entry.context)`. Within the function it is accesses properties/methods `includes`; passed into `normalize`; assigned or updated later in the function.

text: text stores a computed value for temporary calculation. It is initialized from `entry.normalizedText || ""`. Within the function it is accesses properties/methods `includes`; assigned or updated later in the function.

words: words stores a computed value for temporary calculation. It is initialized from `normalizedQuery.split(/\s+/).filter(Boolean)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

score: score stores a numeric value for temporary calculation. It is initialized from 0. Within the function it is returned to the caller; assigned or updated later in the function.

function entryMatches(entry, query)

Purpose and intuition: entryMatches belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: entry is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. query is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with entryScore. Those calls show that entryMatches is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without entryMatches, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function filelsBrowserSearchable(url = "")

Purpose and intuition: filelsBrowserSearchable belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: url points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means filelsBrowserSearchable performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without filelsBrowserSearchable, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from `String(url || "").split(/[?#]/)[0].toLowerCase()`. Within the function it is passed into test; assigned or updated later in the function.

async function indexSearchableFileText(entry)

Purpose and intuition: indexSearchableFileText belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: entry is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: Inside the body, it waits for asynchronous work and calls a network resource. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: performs a fetch and therefore must handle network delay and failure.

What it calls or works with: It works with `fileIsBrowserSearchable`, `flattenSearchText`, `normalize`, `updateSearchDropdown`. Those calls show that `indexSearchableFileText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `indexSearchableFileText`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

response: response stores data returned from a network call. It is initialized from `await fetch(entry.url)`. Within the function it is accesses properties/methods `ok`, `text`; assigned or updated later in the function.

text: text stores the completed result of an awaited operation. It is initialized from `await response.text()`. Within the function it is accesses properties/methods `slice`; passed into `flattenSearchText`, `normalize`; assigned or updated later in the function.

function addProjectSearchEntries(entries, project)

Purpose and intuition: `addProjectSearchEntries` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: entries is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: It works with `slugLabel`, `richTextTerms`, `addSearchEntry`, `sectionHasRenderableContent`, `displayTitle`, `parsedItemTerms`, `fileNameFromUrl`, `itemUrl`. Those calls show that `addProjectSearchEntries` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `addProjectSearchEntries`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

categoryLabel: categoryLabel stores a computed value for portfolio data state. It is initialized from slugLabel(project.category). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into addSearchEntry, electronicsSearchKeywords; assigned or updated later in the function.

baseContext: baseContext stores a string value for temporary calculation. It is initialized from `\${categoryLabel} / \${project.title}`. Within the function it is passed into addSearchEntry; assigned or updated later in the function.

projectTerms: projectTerms stores a list for portfolio data state. It is initialized from []. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into addSearchEntry; assigned or updated later in the function.

sections: sections stores a function or callback value for portfolio data state. It is initialized from (project.portfolioView?.sections || []).filter((section) => section.id !== "brief" && sectionHasRenderableContent(section)). Within the function it is iterated or transformed as a collection; accesses properties/methods forEach; assigned or updated later in the function.

walkItems: walkItems stores a function or callback value for temporary calculation. It is initialized from (items = [], path = [], parentTitles = []) => {. Within the function it is assigned or updated later in the function.

nextPath: nextPath stores a list for filesystem location. It is initialized from [...path, index]. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into pathToString; assigned or updated later in the function.

title: title stores a computed value for temporary calculation. It is initialized from displayTitle(item.title || item.name || item.label || fileNameFromUrl(item.url || ""), "Project item"). Within the function it is passed into addSearchEntry, displayTitle; assigned or updated later in the function.

url: url stores a computed value for network or routing value. It is initialized from itemUrl(item). Within the function it is passed into displayTitle, fileNameFromUrl; assigned or updated later in the function.

context: context stores a list for temporary calculation. It is initialized from [baseContext, displayTitle(section.title, "Section"), ...parentTitles].filter(Boolean).join(" / "). Within the function it is passed into addSearchEntry; assigned or updated later in the function.

entry: entry stores a structured object for temporary calculation. It is initialized from {. Within the function it is passed into addSearchEntry; assigned or updated later in the function.

storedEntry: storedEntry stores a computed value for temporary calculation. It is initialized from addSearchEntry(entries, entry). Within the function it is passed into indexSearchableFileText; assigned or updated later in the function.

entry: entry stores a structured object for temporary calculation. It is initialized from {. Within the function it is passed into addSearchEntry; assigned or updated later in the function.

storedEntry: storedEntry stores a computed value for temporary calculation. It is initialized from addSearchEntry(entries, entry). Within the function it is passed into indexSearchableFileText; assigned or updated later in the function.

function addSiteSectionSearchEntries(entries)

Purpose and intuition: addSiteSectionSearchEntries belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: entries is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: It works with normalizeProfile, addSearchEntry, indexSearchableFileText, richTextTerms. Those calls show that addSiteSectionSearchEntries is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without addSiteSectionSearchEntries, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

current: current stores a computed value for temporary calculation. It is initialized from normalizeProfile(profile || {}). Within the function it accesses properties/methods contactIntro, displayName, email, githubUrl, linkedinUrl, phone, portfolioLabel, resumeUrl; passed into addSearchEntry; assigned or updated later in the function.

pageSections: pageSections stores a list for portfolio data state. It is initialized from [. Within the function it is iterated or transformed as a collection; accesses properties/methods forEach, splice; assigned or updated later in the function.

resumeTextEntry: resumeTextEntry stores a computed value for temporary calculation. It is initialized from addSearchEntry(entries, {. Within the function it is passed into indexSearchableFileText; assigned or updated later in the function.

function rebuildSearchIndex()

Purpose and intuition: rebuildSearchIndex belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: It works with addSiteSectionSearchEntries, addProjectSearchEntries. Those calls show that rebuildSearchIndex is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `rebuildSearchIndex`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

entries: `entries` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is passed into `addProjectSearchEntries`, `addSiteSectionSearchEntries`; assigned or updated later in the function.

function searchResultsFor(query)

Purpose and intuition: `searchResultsFor` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `query` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: It works with `normalize`, `entryScore`. Those calls show that `searchResultsFor` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `searchResultsFor`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

normalizedQuery: `normalizedQuery` stores a computed value for temporary calculation. It is initialized from `normalize(query).trim()`. Within the function it is passed into `entryScore`; assigned or updated later in the function.

seen: `seen` stores a constructed object for lookup collection. It is initialized from `new Set()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `add`, `has`; assigned or updated later in the function.

key: `key` stores a list for temporary calculation. It is initialized from `[entry.kind, entry.projectId, entry.sectionIndex, entry.resourcePath, entry.domTarget, entry.url, entry.title].join("|")`. Within the function it is passed into `add`; assigned or updated later in the function.

function highlightQueryHtml(value = "", query = "")

Purpose and intuition: `highlightQueryHtml` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `query` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing,

or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `escapeHtml`, `escapeRegExp`. Those calls show that `highlightQueryHtml` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `highlightQueryHtml`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

escaped: `escaped` stores a computed value for temporary calculation. It is initialized from `escapeHtml(value)`. Within the function it is returned to the caller; accesses properties/methods `replace`; assigned or updated later in the function.

cleanQuery: `cleanQuery` stores a computed value for temporary calculation. It is initialized from `String(query || "").trim()`. Within the function it is passed into `RegExp`; assigned or updated later in the function.

pattern: `pattern` stores a constructed object for temporary calculation. It is initialized from `new RegExp(`${escapeRegExp(escapeHtml(cleanQuery))}`, "ig")`. Within the function it is passed into `replace`; assigned or updated later in the function.

function clearSearchHighlights()

Purpose and intuition: `clearSearchHighlights` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: No other named helper from this file is detected inside the body. That means `clearSearchHighlights` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `clearSearchHighlights`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function textNodeSearchTargets()

Purpose and intuition: textNodeSearchTargets belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser DOM. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: No other named helper from this file is detected inside the body. That means textNodeSearchTargets performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without textNodeSearchTargets, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function applySearchHighlights(query)

Purpose and intuition: applySearchHighlights belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: query is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with clearSearchHighlights, normalize, textNodeSearchTargets. Those calls show that applySearchHighlights is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without applySearchHighlights, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

cleanQuery: cleanQuery stores a computed value for temporary calculation. It is initialized from String(query || "").trim(). Within the function it is accesses properties/methods length; passed into indexOf, normalize, setEnd; assigned or updated later in the function.

ranges: ranges stores a list for temporary calculation. It is initialized from []. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods length, push; passed into set; assigned or updated later in the function.

needle: needle stores a computed value for temporary calculation. It is initialized from normalize(cleanQuery). Within the function it is passed into includes, indexOf; assigned or updated later in the function.

walker: walker stores a DOM element reference. It is initialized from `document.createTreeWalker(root, NodeFilter.SHOW_TEXT, {`. Within the function it is accesses properties/methods `nextNode`; assigned or updated later in the function.

node: node stores a computed value for temporary calculation. It is initialized from `walker.nextNode()`. Within the function it is accesses properties/methods `parentElement`, `textContent`; passed into `createTreeWalker`, `normalize`, `setEnd`, `setStart`; assigned or updated later in the function.

haystack: haystack stores a computed value for runtime state or cache. It is initialized from `normalize(node.textContent)`. Within the function it is accesses properties/methods `indexOf`; assigned or updated later in the function.

index: index stores a computed value for temporary calculation. It is initialized from `haystack.indexOf(needle)`. Within the function it is passed into `indexOf`, `setEnd`, `setStart`; assigned or updated later in the function.

range: range stores a DOM element reference. It is initialized from `document.createRange()`. Within the function it is accesses properties/methods `setEnd`, `setStart`; passed into `push`; assigned or updated later in the function.

function queueSearchHighlights()

Purpose and intuition: `queueSearchHighlights` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it uses a timer to avoid blocking or to wait for another process. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: It works with `applySearchHighlights`. Those calls show that `queueSearchHighlights` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `queueSearchHighlights`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function ensureSearchPanel()

Purpose and intuition: `ensureSearchPanel` creates or repairs something only when it is needed. This pattern avoids duplicate DOM nodes, duplicate repositories, duplicate handlers, or missing folders while still letting later code assume the resource exists.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: creates DOM elements instead of relying only on static HTML; writes generated markup into the page, which is why escaping and sanitizing helpers around it matter; attaches event handlers that turn user actions into behavior.

What it calls or works with: It works with `updateSearchDropdown`, `goToSearchResult`. Those calls show that `ensureSearchPanel` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `ensureSearchPanel`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

shell: `shell` stores a DOM element reference. It is initialized from `document.createElement("div")`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `append`, `className`; passed into `replaceWith`; assigned or updated later in the function.

button: `button` stores a computed value for DOM reference or UI state. It is initialized from `event.target.closest("[data-search-result-index]")`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods `dataset`; passed into `Number`; assigned or updated later in the function.

result: `result` stores a computed value for temporary calculation. It is initialized from `currentSearchResults[Number(button.dataset.searchResultIndex)]`. Within the function it is passed into `closest`, `goToSearchResult`, `querySelector`; assigned or updated later in the function.

function `setSearchMessage(message, state = "neutral")`

Purpose and intuition: `setSearchMessage` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `message` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `state` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: No other named helper from this file is detected inside the body. That means `setSearchMessage` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `setSearchMessage`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function renderSearchResults(query, visibleCount)

Purpose and intuition: renderSearchResults turns saved data for SearchResults into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that styles.css expects.

Parameters and data it receives: query is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. visibleCount is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

What it calls or works with: It works with ensureSearchPanel, searchResultsFor, setSearchMessage, highlightQueryHtml, escapeHtml, searchSnippet. Those calls show that renderSearchResults is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without renderSearchResults, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

list: list stores a DOM element reference. It is initialized from searchPanel.querySelector(".search-results-list"). Within the function it is accesses properties/methods innerHTML; passed into querySelector; assigned or updated later in the function.

cleanQuery: cleanQuery stores a computed value for temporary calculation. It is initialized from String(query || "").trim(). Within the function it is passed into highlightQueryHtml, searchResultsFor, searchSnippet, setAttribute, setSearchMessage; assigned or updated later in the function.

limitedResults: limitedResults stores a computed value for temporary calculation. It is initialized from currentSearchResults.slice(0, searchResultLimit). Within the function it is iterated or transformed as a collection; accesses properties/methods length, map; passed into setSearchMessage; assigned or updated later in the function.

visibleText: visibleText stores a computed value for temporary calculation. It is initialized from visibleCount. Within the function it is passed into setSearchMessage; assigned or updated later in the function.

function updateSearchDropdown()

Purpose and intuition: updateSearchDropdown belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: It works with `applySearchHighlights`, `renderSearchResults`. Those calls show that `updateSearchDropdown` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `updateSearchDropdown`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

query: `query` stores a computed value for temporary calculation. It is initialized from `searchInput?.value || ""`. Within the function it is passed into `applySearchHighlights`, `renderSearchResults`; assigned or updated later in the function.

visibleCount: `visibleCount` stores a computed value for temporary calculation. It is initialized from `applySearchHighlights(query)`. Within the function it is passed into `renderSearchResults`; assigned or updated later in the function.

function showSearchPanelIfNeeded()

Purpose and intuition: `showSearchPanelIfNeeded` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: It works with `updateSearchDropdown`. Those calls show that `showSearchPanelIfNeeded` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `showSearchPanelIfNeeded`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function goToSearchResult(result)

Purpose and intuition: `goToSearchResult` turns a search dropdown item into navigation. Depending on what the result represents, it may open a project window, jump to a main page section, highlight text on the current page, or open a file/link. This makes search feel like a search engine instead of a passive list.

Purpose and intuition: The function is necessary because search results can point to many different kinds of things: project titles, section overviews, files, custom sections, contact links, or page text. One navigation rule would not work for all of them.

Parameters and data it receives: `result` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: It works with `scrollToDomTarget`, `openParsedSection`, `queueSearchHighlights`, `setActiveFilter`, `renderProjects`. Those calls show that `goToSearchResult` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `goToSearchResult`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

target: target stores a DOM element reference. It is initialized from `document.getElementById(result.projectId)`. Within the function it is assigned or updated later in the function.

function handleSearchInput()

Purpose and intuition: `handleSearchInput` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: It works with `renderProjects`, `updateSearchDropdown`. Those calls show that `handleSearchInput` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `handleSearchInput`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

Portfolio AI frontend

The functions in this group all support portfolio ai frontend. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function assistantEndpoint()

Purpose and intuition: `assistantEndpoint` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser DOM. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: No other named helper from this file is detected inside the body. That means assistantEndpoint performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without assistantEndpoint, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

metaEndpoint: metaEndpoint stores a DOM element reference. It is initialized from `document.querySelector('meta[name="portfolio-ai-endpoint"]')?.content || ""`. Within the function it is accesses properties/methods `trim`; passed into `String`; assigned or updated later in the function.

function assistantSourceLabel(result = {})

Purpose and intuition: assistantSourceLabel is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: result is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: No other named helper from this file is detected inside the body. That means assistantSourceLabel performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without assistantSourceLabel, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

title: title stores a computed value for temporary calculation. It is initialized from `result.title || "Portfolio item"`. Within the function it is returned to the caller; assigned or updated later in the function.

contextParts: contextParts stores a computed value for temporary calculation. It is initialized from `String(result.context || "")`. Within the function it is accesses properties/methods `slice`; assigned or updated later in the function.

specificContext: specificContext stores a resolved filesystem or route value. It is initialized from `contextParts.slice(-2).join(" / ")`. Within the function it is assigned or updated later in the function.

function assistantProjectSummary(project = {})

Purpose and intuition: assistantProjectSummary is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: It works with slugLabel, sectionHasRenderableContent, displayTitle. Those calls show that assistantProjectSummary is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without assistantProjectSummary, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

categoryLabel: categoryLabel stores a computed value for portfolio data state. It is initialized from slugLabel(project.category). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

sections: sections stores a function or callback value for portfolio data state. It is initialized from (project.portfolioView?.sections || []). Within the function it is assigned or updated later in the function.

function assistantProjectTitleAcronyms(title = "")

Purpose and intuition: assistantProjectTitleAcronyms is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: title is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: It works with normalize. Those calls show that assistantProjectTitleAcronyms is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without assistantProjectTitleAcronyms, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

words: words stores a computed value for temporary calculation. It is initialized from `normalize(title)`. Within the function it is accesses properties/methods `length`, `slice`; passed into `add`; assigned or updated later in the function.

acronyms: acronyms stores a constructed object for lookup collection. It is initialized from `new Set()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `add`; assigned or updated later in the function.

start: start stores a numeric value for temporary calculation. It is initialized from 0. Within the function it is passed into `add`; assigned or updated later in the function.

length: length stores a numeric value for temporary calculation. It is initialized from 2. Within the function it is passed into `add`; assigned or updated later in the function.

function `assistantProjectIsNamedInQuestion(question = "", project = {})`

Purpose and intuition: `assistantProjectIsNamedInQuestion` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `normalize`, `assistantProjectTitleAcronyms`. Those calls show that `assistantProjectIsNamedInQuestion` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `assistantProjectIsNamedInQuestion`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is accesses properties/methods `includes`, `split`; passed into `Set`; assigned or updated later in the function.

fillerWords: `fillerWords` stores a constructed object for lookup collection. It is initialized from `new Set(["with", "from", "into", "using", "project", "design", "system", "systems"])`. Within the function it is accesses properties/methods `has`; assigned or updated later in the function.

rawTitle: `rawTitle` stores a computed value for temporary calculation. It is initialized from `project.portfolioView?.title || project.title || project.id`. Within the function it is passed into `assistantProjectTitleAcronyms`, `normalize`; assigned or updated later in the function.

title: `title` stores a computed value for temporary calculation. It is initialized from `normalize(rawTitle)`. Within the function it is accesses properties/methods `split`; assigned or updated later in the function.

cleanWords: cleanWords stores a constructed object for lookup collection. It is initialized from new Set(clean.split(/\s+/)). Within the function it is accesses properties/methods has; assigned or updated later in the function.

acronymMatch: acronymMatch stores a computed value for temporary calculation. It is initialized from assistantProjectTitleAcronyms(rawTitle). Within the function it is assigned or updated later in the function.

words: words stores a function or callback value for temporary calculation. It is initialized from title.split(/\s+/).filter((word) => word.length > 3 && !fillerWords.has(word)). Within the function it is returned to the caller; iterated or transformed as a collection; accesses properties/methods filter, length; passed into min; assigned or updated later in the function.

matches: matches stores a function or callback value for temporary calculation. It is initialized from words.filter((word) => clean.includes(word)).length. Within the function it is assigned or updated later in the function.

function assistantNamedProjectIds(question = "")

Purpose and intuition: assistantNamedProjectIds is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: question is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with assistantProjectIsNamedInQuestion. Those calls show that assistantNamedProjectIds is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without assistantNamedProjectIds, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: No local const, let, or var values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function assistantProjectTitleMatches(question = "")

Purpose and intuition: assistantProjectTitleMatches is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: question is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `assistantNamedProjectIds`. Those calls show that `assistantProjectTitleMatches` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `assistantProjectTitleMatches`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function `assistantUrlKind(url = "")`

Purpose and intuition: `assistantUrlKind` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `fileIsBrowserSearchable`. Those calls show that `assistantUrlKind` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `assistantUrlKind`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(url || "").split(/[?#]/)[0].toLowerCase()`. Within the function it is passed into `test`; assigned or updated later in the function.

function `assistantAbsoluteUrl(url = "")`

Purpose and intuition: `assistantAbsoluteUrl` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `normalizeLinkTarget`, `looksLikeBareWebAddress`. Those calls show that `assistantAbsoluteUrl` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `assistantAbsoluteUrl`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

target: `target` stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(url, { assumeWeb: looksLikeBareWebAddress(url) || /^https?:\W/i.test(url) })`. Within the function it is returned to the caller; passed into `URL`; assigned or updated later in the function.

function assistantLinkRecordsFromValue(value, context = "")

Purpose and intuition: `assistantLinkRecordsFromValue` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `context` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `uniqueDownloads`, `collectDownloadsFromValue`, `assistantUrlKind`, `fileNameFromUrl`, `assistantAbsoluteUrl`. Those calls show that `assistantLinkRecordsFromValue` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `assistantLinkRecordsFromValue`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

records: `records` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `push`; assigned or updated later in the function.

downloads: `downloads` stores a computed value for temporary calculation. It is initialized from `uniqueDownloads(collectDownloadsFromValue(value, "Portfolio asset", []))`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

rawUrl: `rawUrl` stores a computed value for network or routing value. It is initialized from `download.url || download.href || download.path || ""`. Within the function it is passed into `assistantAbsoluteUrl`, `fileNameFromUrl`, `push`; assigned or updated later in the function.

function assistantUrlsFromTextValue(value, context = "")

Purpose and intuition: assistantUrlsFromTextValue is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. context is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: It works with flattenSearchText, assistantUrlKind, fileNameFromUrl, assistantAbsoluteUrl. Those calls show that assistantUrlsFromTextValue is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without assistantUrlsFromTextValue, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

text: text stores a computed value for temporary calculation. It is initialized from flattenSearchText([value]). Within the function it is accesses properties/methods match; assigned or updated later in the function.

matches: matches stores a computed value for temporary calculation. It is initialized from text.match(/b(?:https?:VV|www\.)[^s<>"])+/gi) || []. Within the function it is iterated or transformed as a collection; accesses properties/methods map; passed into Set; assigned or updated later in the function.

function assistantPublicProfileLinks()

Purpose and intuition: assistantPublicProfileLinks is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with fileNameFromUrl, assistantUrlKind, assistantAbsoluteUrl, normalizeProfile. Those calls show that assistantPublicProfileLinks is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without assistantPublicProfileLinks, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

records: records stores a list for temporary calculation. It is initialized from []. Within the function it is returned to the caller; iterated or transformed as a collection; mutated as a collection or DOM container; accesses properties/methods filter, push; assigned or updated later in the function.

add: add stores a function or callback value for temporary calculation. It is initialized from (item = {}, context = "") => {}. Within the function it is assigned or updated later in the function.

rawUrl: rawUrl stores a computed value for network or routing value. It is initialized from item.url || item.href || item.path || "". Within the function it is passed into assistantAbsoluteUrl, fileNameFromUrl, push; assigned or updated later in the function.

label: label stores a computed value for temporary calculation. It is initialized from item.label || item.title || fileNameFromUrl(rawUrl). Within the function it is passed into add; assigned or updated later in the function.

current: current stores a computed value for temporary calculation. It is initialized from normalizeProfile(profile || {}). Within the function it is accesses properties/methods githubUrl, linkedinUrl, resumeUrl, websiteUrl; passed into add; assigned or updated later in the function.

function assistantProjectEvidence(project = {})

Purpose and intuition: assistantProjectEvidence is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: It works with uniqueDownloads, collectDownloadsFromValue, assistantUrlKind, fileNameFromUrl, assistantAbsoluteUrl, assistantUrlsFromTextValue. Those calls show that assistantProjectEvidence is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without assistantProjectEvidence, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

downloads: downloads stores a computed value for temporary calculation. It is initialized from uniqueDownloads(collectDownloadsFromValue(project, "Project asset", [])). Within the function it is accesses properties/methods slice; assigned or updated later in the function.

downloadRecords: downloadRecords stores a function or callback value for lookup collection. It is initialized from downloads.slice(0, 26).map((download) => {}. Within the function it is assigned or updated later in the function.

rawUrl: rawUrl stores a computed value for network or routing value. It is initialized from download.url || "". Within the function it is passed into assistantAbsoluteUrl, assistantUrlKind, fileNameFromUrl; assigned or updated later in the function.

textLinkRecords: textLinkRecords stores a computed value for temporary calculation. It is initialized from assistantUrlsFromTextValue([project.summary, project.links, project.portfolioView], project.portfolioView?.title || project.title). Within the function it is assigned or updated later in the function.

function assistantQuestionIsCasual(question = "")

Purpose and intuition: assistantQuestionIsCasual is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: question is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with normalize. Those calls show that assistantQuestionIsCasual is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without assistantQuestionIsCasual, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from normalize(question). Within the function it is passed into test; assigned or updated later in the function.

function assistantQuestionHasPortfolioIntent(question = "")

Purpose and intuition: assistantQuestionHasPortfolioIntent is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: question is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with normalize, assistantProjectTitleMatches. Those calls show that assistantQuestionHasPortfolioIntent is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without assistantQuestionHasPortfolioIntent, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

function assistantConversationSuggestsPortfolioContext(history = [])

Purpose and intuition: `assistantConversationSuggestsPortfolioContext` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: `history` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `normalize`. Those calls show that `assistantConversationSuggestsPortfolioContext` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `assistantConversationSuggestsPortfolioContext`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function assistantQuestionAllowsPublicLookup(question = "", intent = assistantQuestionIntent(question))

Purpose and intuition: `assistantQuestionAllowsPublicLookup` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `intent` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `normalize`. Those calls show that `assistantQuestionAllowsPublicLookup` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `assistantQuestionAllowsPublicLookup`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into test; assigned or updated later in the function.

function assistantQuestionLooksConceptual(question = "")

Purpose and intuition: `assistantQuestionLooksConceptual` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `normalize`, `assistantQuestionIsEngineeringRelated`. Those calls show that `assistantQuestionLooksConceptual` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `assistantQuestionLooksConceptual`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into test; assigned or updated later in the function.

function assistantQuestionIntent(question = "", history = assistantChatHistory)

Purpose and intuition: `assistantQuestionIntent` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `history` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `assistantQuestionIsCasual`, `normalize`, `assistantQuestionHasPortfolioIntent`, `assistantConversationSuggestsPortfolioContext`, `assistantQuestionIsEngineeringRelated`, `assistantQuestionLooksConceptual`. Those calls show that `assistantQuestionIntent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `assistantQuestionIntent`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

hasPortfolioIntent: `hasPortfolioIntent` stores a computed value for temporary calculation. It is initialized from `assistantQuestionHasPortfolioIntent(question)`. Within the function it is assigned or updated later in the function.

function assistantQuestionTokens(question = "")

Purpose and intuition: `assistantQuestionTokens` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `normalize`. Those calls show that `assistantQuestionTokens` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `assistantQuestionTokens`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

stopWords: `stopWords` stores a constructed object for lookup collection. It is initialized from `new Set([])`. Within the function it is accesses properties/methods has; assigned or updated later in the function.

function assistantSpecificSourceTokens(question = "")

Purpose and intuition: `assistantSpecificSourceTokens` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `assistantQuestionTokens`. Those calls show that `assistantSpecificSourceTokens` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `assistantSpecificSourceTokens`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

genericTokens: `genericTokens` stores a constructed object for lookup collection. It is initialized from `new Set([`. Within the function it is accesses properties/methods `has`; assigned or updated later in the function.

function assistantPromptTargetsProjectLanding(question = "")

Purpose and intuition: `assistantPromptTargetsProjectLanding` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `normalize`. Those calls show that `assistantPromptTargetsProjectLanding` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `assistantPromptTargetsProjectLanding`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

function assistantEntryRelationScore(result = {}, question = "")

Purpose and intuition: `assistantEntryRelationScore` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: `result` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `assistantSpecificSourceTokens`, `normalize`. Those calls show that `assistantEntryRelationScore` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `assistantEntryRelationScore`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

tokens: `tokens` stores a computed value for temporary calculation. It is initialized from `assistantSpecificSourceTokens(question)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `length`; assigned or updated later in the function.

haystack: `haystack` stores a resolved filesystem or route value. It is initialized from `normalize([result.title, result.context, result.type, result.text, result.url].filter(Boolean).join(" "))`. Within the function it is accesses properties/methods `includes`; assigned or updated later in the function.

tokenHits: `tokenHits` stores a function or callback value for temporary calculation. It is initialized from `tokens.filter((token) => haystack.includes(token)).length`. Within the function it is assigned or updated later in the function.

relation: `relation` stores a computed value for temporary calculation. It is initialized from `tokenHits * 30`. Within the function it is returned to the caller; assigned or updated later in the function.

function `assistantNamedProjectTopicTokens(question = "", namedIds = assistantNamedProjectIds(question))`

Purpose and intuition: `assistantNamedProjectTopicTokens` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `namedIds` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `normalize`, `assistantProjectTitleAcronyms`, `assistantSpecificSourceTokens`. Those calls show that `assistantNamedProjectTopicTokens` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `assistantNamedProjectTopicTokens`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

projectVocabulary: projectVocabulary stores a constructed object for portfolio data state. It is initialized from new Set(). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is mutated as a collection or DOM container; accesses properties/methods add, has; assigned or updated later in the function.

title: title stores a computed value for temporary calculation. It is initialized from project.portfolioView?.title || project.title || project.id. Within the function it is passed into assistantProjectTitleAcronyms, normalize; assigned or updated later in the function.

function assistantSourcesForDisplay(question = "", results = [], intent = assistantQuestionIntent(question))

Purpose and intuition: assistantSourcesForDisplay chooses which portfolio links appear under an assistant answer. The user complained earlier that related links were generic; this function is the fix-point for that behavior. It scores source relation to the prompt, respects named projects and topic tokens, and avoids showing source buttons that do not belong to the answer context.

Purpose and intuition: The function matters because sources are a promise. A displayed link should help verify or explore the answer. If this function is too loose, the assistant looks random even when the text answer is good.

Parameters and data it receives: question is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. results is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. intent is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: It works with assistantSpecificSourceTokens, assistantNamedProjectIds, assistantNamedProjectTopicTokens, normalize, assistantPromptTargetsProjectLanding, assistantEntryRelationScore. Those calls show that assistantSourcesForDisplay is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without assistantSourcesForDisplay, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

tokens: tokens stores a computed value for temporary calculation. It is initialized from assistantSpecificSourceTokens(question). Within the function it is iterated or transformed as a collection; accesses properties/methods filter, length; assigned or updated later in the function.

namedProjectIdList: namedProjectIdList stores a computed value for portfolio data state. It is initialized from assistantNamedProjectIds(question). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into Set, assistantNamedProjectTopicTokens; assigned or updated later in the function.

namedProjectIds: namedProjectIds stores a constructed object for portfolio data state. It is initialized from new Set(namedProjectIdList). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods has, size; assigned or updated later in the function.

namedProjectTopicTokens: namedProjectTopicTokens stores a computed value for portfolio data state. It is initialized from assistantNamedProjectTopicTokens(question, namedProjectIdList). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is iterated or transformed as a collection; accesses properties/methods filter, length; assigned or updated later in the function.

asksForLinks: asksForLinks stores a computed value for temporary calculation. It is initialized from `\b(open|show|where|link|links|github|linkedin|resume|download|file|files|repo|repository|source\s+code)\b/.test(normalize(question))`. Within the function it is passed into slice; assigned or updated later in the function.

asksForProjectLanding: asksForProjectLanding stores a computed value for portfolio data state. It is initialized from assistantPromptTargetsProjectLanding(question). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

pageLinkIntent: pageLinkIntent stores a computed value for temporary calculation. It is initialized from `\b(resume|github|linkedin|contact|email|phone)\b/.test(normalize(question))`. Within the function it is assigned or updated later in the function.

asksForSpecificArtifact: asksForSpecificArtifact stores a computed value for temporary calculation. It is initialized from `\b(code|source|repo|repository|github|file|files|document|documents|test|tests|result|results|pcb|schematic|simulation|tool|tools)\b/.test(normalize(question))`. Within the function it is assigned or updated later in the function.

filtered: filtered stores a computed value for temporary calculation. It is initialized from results. Within the function it is returned to the caller; assigned or updated later in the function.

haystack: haystack stores a resolved filesystem or route value. It is initialized from `normalize([result.title, result.context, result.type, result.text, result.url].filter(Boolean).join(" ")). Within the function it is accesses properties/methods includes; assigned or updated later in the function.`

tokenHits: tokenHits stores a function or callback value for temporary calculation. It is initialized from `tokens.filter((token) => haystack.includes(token)).length`. Within the function it is assigned or updated later in the function.

topicHits: topicHits stores a function or callback value for temporary calculation. It is initialized from `namedProjectTopicTokens.filter((token) => haystack.includes(token)).length`. Within the function it is assigned or updated later in the function.

function assistantContextForQuestion(question = "", intent = assistantQuestionIntent(question), knownResults = null)

Purpose and intuition: assistantContextForQuestion builds the packet of portfolio evidence sent to the AI backend. It uses the question intent, search results, named project matches, profile links, project summaries, and available evidence to decide what the model should see. It is selective by design so a generic greeting does not receive a pile of unrelated project context.

Purpose and intuition: Without this function, the assistant would either know too little about the portfolio or always overload the model with the same generic links. This is the function that makes the AI choose context based on the conversation.

Parameters and data it receives: question is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. intent is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. knownResults is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

What it calls or works with: It works with searchResultsFor, assistantNamedProjectIds, normalize, assistantPublicProfileLinks, assistantUrlKind, assistantAbsoluteUrl, assistantProjectEvidence, normalizeProfile. Those calls show that assistantContextForQuestion is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without assistantContextForQuestion, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

directResults: directResults stores a list for filesystem location. It is initialized from ["general_engineering", "general_knowledge", "general_conversation"].includes(intent). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is iterated or transformed as a collection; accesses properties/methods map; passed into Set; assigned or updated later in the function.

projectIds: projectIds stores a list for portfolio data state. It is initialized from [...new Set([. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

matchedProjects: matchedProjects stores a computed value for portfolio data state. It is initialized from projectIds. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is iterated or transformed as a collection; accesses properties/methods flatMap, map; assigned or updated later in the function.

includePortfolioLinkContext: includePortfolioLinkContext stores a computed value for temporary calculation. It is initialized from intent === "portfolio_specific". Within the function it is assigned or updated later in the function.

wantsPublicCode: wantsPublicCode stores a computed value for temporary calculation. It is initialized from `\b(github|repo|repository|repositories|source|s+code|code|snippet|firmware|verilog|systemverilog|python|javascript)\b/.test(normalize(question))`. Within the function it is assigned or updated later in the function.

publicProfileFetches: publicProfileFetches stores a computed value for portfolio data state. It is initialized from includePortfolioLinkContext. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

aGithub: aGithub stores a computed value for Git publishing and authorization state. It is initialized from /github/i.test(`\${a.kind || ""} \${a.url || ""}`) ? 1 : 0. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is assigned or updated later in the function.

bGithub: bGithub stores a computed value for Git publishing and authorization state. It is initialized from /github/i.test(`\${b.kind || ""} \${b.url || ""}`) ? 1 : 0. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is returned to the caller; assigned or updated later in the function.

resultFetches: resultFetches stores a function or callback value for lookup collection. It is initialized from directResults.map((result) => ({). Within the function it is assigned or updated later in the function.

projectFetches: projectFetches stores a function or callback value for portfolio data state. It is initialized from matchedProjects.flatMap((project) => assistantProjectEvidence(project). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

sourceFetches: sourceFetches stores a list for temporary calculation. It is initialized from [. Within the function it is iterated or transformed as a collection; accesses properties/methods filter; assigned or updated later in the function.

uniqueSourceFetches: uniqueSourceFetches stores a function or callback value for temporary calculation. It is initialized from sourceFetches.filter((item, index, array) => (. Within the function it is assigned or updated later in the function.

function assistantRemember(role, content = "")

Purpose and intuition: assistantRemember is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: role is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. content is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: No other named helper from this file is detected inside the body. That means assistantRemember performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without assistantRemember, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

cleanRole: cleanRole stores a computed value for temporary calculation. It is initialized from role === "assistant" ? "assistant" : "user". Within the function it is passed into push; assigned or updated later in the function.

cleanContent: cleanContent stores a computed value for temporary calculation. It is initialized from String(content || "").trim(). Within the function it is accesses properties/methods slice; passed into push; assigned or updated later in the function.

function assistantConversationForRequest(currentQuestion = "", priorConversation = assistantChatHistory)

Purpose and intuition: assistantConversationForRequest is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: currentQuestion is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. priorConversation is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it updates browser route/history. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means assistantConversationForRequest performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without assistantConversationForRequest, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

history: history stores a function or callback value for runtime state or cache. It is initialized from priorConversation.slice(-8).map((item) => ({). Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods push; assigned or updated later in the function.

function assistantFallbackResults(question = "", intent = assistantQuestionIntent(question))

Purpose and intuition: assistantFallbackResults is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: question is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. intent is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: It works with `normalize`, `assistantNamedProjectIds`, `searchResultsFor`, `slugLabel`, `flattenSearchText`. Those calls show that `assistantFallbackResults` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `assistantFallbackResults`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question).trim()`. Within the function it accesses properties/methods `includes`; passed into `searchResultsFor`; assigned or updated later in the function.

namedProjectIds: `namedProjectIds` stores a computed value for portfolio data state. It is initialized from `assistantNamedProjectIds(question)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is returned to the caller; accesses properties/methods `includes`, `length`; assigned or updated later in the function.

results: `results` stores a computed value for temporary calculation. It is initialized from `searchResultsFor(clean)`. Within the function it is returned to the caller; iterated or transformed as a collection; accesses properties/methods `filter`, `length`, `slice`; assigned or updated later in the function.

focusedResults: `focusedResults` stores a function or callback value for temporary calculation. It is initialized from `results.filter((result) => namedProjectIds.includes(result.projectId))`. Within the function it is returned to the caller; accesses properties/methods `length`, `slice`; assigned or updated later in the function.

category: `category` stores a function or callback value for portfolio data state. It is initialized from `categories.find((item) => clean.includes(normalize(item.label)) || clean.includes(normalize(item.id)))`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it accesses properties/methods `id`, `label`; passed into `slugLabel`; assigned or updated later in the function.

function `assistantQuestionIsEngineeringRelated(question = "")`

Purpose and intuition: `assistantQuestionIsEngineeringRelated` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `normalize`. Those calls show that `assistantQuestionIsEngineeringRelated` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `assistantQuestionIsEngineeringRelated`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into test; assigned or updated later in the function.

function assistantGeneralEngineeringAnswer(question = "")

Purpose and intuition: `assistantGeneralEngineeringAnswer` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: It works with `normalize`, `assistantQuestionIsEngineeringRelated`. Those calls show that `assistantGeneralEngineeringAnswer` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `assistantGeneralEngineeringAnswer`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into test; assigned or updated later in the function.

function assistantLocalAnswer(question = "", results = [], intent = assistantQuestionIntent(question))

Purpose and intuition: `assistantLocalAnswer` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `results` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `intent` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

What it calls or works with: It works with `normalize`, `normalizeProfile`, `assistantGeneralEngineeringAnswer`, `searchSnippet`. Those calls show that `assistantLocalAnswer` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `assistantLocalAnswer`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

ownerName: `ownerName` stores a computed value for temporary calculation. It is initialized from `normalizeProfile(profile || {}).displayName || "the portfolio owner"`. Within the function it is assigned or updated later in the function.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

generalAnswer: `generalAnswer` stores a computed value for temporary calculation. It is initialized from `assistantGeneralEngineeringAnswer(question)`. Within the function it is returned to the caller; assigned or updated later in the function.

generalAnswer: `generalAnswer` stores a computed value for temporary calculation. It is initialized from `assistantGeneralEngineeringAnswer(question)`. Within the function it is returned to the caller; assigned or updated later in the function.

projectIds: `projectIds` stores a list for portfolio data state. It is initialized from `[...new Set(results.map((item) => item.projectId).filter(Boolean))]`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

matchedProjects: `matchedProjects` stores a computed value for portfolio data state. It is initialized from `projectIds`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is iterated or transformed as a collection; accesses properties/methods `map`; assigned or updated later in the function.

projectNames: `projectNames` stores a function or callback value for portfolio data state. It is initialized from `matchedProjects.map((project) => project.portfolioView?.title || project.title).slice(0, 3)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it accesses properties/methods `join`, `length`; passed into `push`; assigned or updated later in the function.

fileCount: `fileCount` stores a function or callback value for temporary calculation. It is initialized from `results.filter((item) => item.kind === "file").length`. Within the function it is passed into `push`; assigned or updated later in the function.

sectionCount: `sectionCount` stores a function or callback value for portfolio data state. It is initialized from `results.filter((item) => item.kind === "section" || item.kind === "subsection").length`. Within the function it is passed into `push`; assigned or updated later in the function.

pageCount: `pageCount` stores a function or callback value for temporary calculation. It is initialized from `results.filter((item) => item.kind === "page").length`. Within the function it is passed into `push`; assigned or updated later in the function.

parts: parts stores a list for temporary calculation. It is initialized from []. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods join, push; assigned or updated later in the function.

firstSnippet: firstSnippet stores a computed value for temporary calculation. It is initialized from searchSnippet(results[0].text, question). Within the function it is passed into push; assigned or updated later in the function.

function assistantSourcesMarkup(results = [])

Purpose and intuition: assistantSourcesMarkup is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: results is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with escapeHtml, assistantSourceLabel. Those calls show that assistantSourcesMarkup is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without assistantSourcesMarkup, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

sources: sources stores a computed value for temporary calculation. It is initialized from results.slice(0, 5). Within the function it is iterated or transformed as a collection; accesses properties/methods length, map; assigned or updated later in the function.

sourceButtons: sourceButtons stores a function or callback value for DOM reference or UI state. It is initialized from sources.map((source) => {. It anchors UI behavior to a specific browser element or window state. Within the function it is assigned or updated later in the function.

id: id stores a computed value for temporary calculation. It is initialized from String(assistantSourceCounter++). Within the function it is passed into set; assigned or updated later in the function.

function setAssistantStatus(message, state = "ready")

Purpose and intuition: setAssistantStatus belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: message is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. state is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or

rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: No other named helper from this file is detected inside the body. That means `setAssistantStatus` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `setAssistantStatus`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function assistantDesktopDockedHeight()

Purpose and intuition: `assistantDesktopDockedHeight` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: No other named helper from this file is detected inside the body. That means `assistantDesktopDockedHeight` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `assistantDesktopDockedHeight`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

value: `value` stores a computed value for temporary calculation. It is initialized from `getComputedStyle(aiAssistantPanel).getPropertyValue("--ai-console-rest-height").trim()`. Within the function it is accessed via `properties/methods endsWith`; passed into `parseFloat`; assigned or updated later in the function.

function updateAssistantPanelGrowth()

Purpose and intuition: `updateAssistantPanelGrowth` belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: toggles CSS classes to express UI state.

What it calls or works with: It works with `assistantDesktopDockedHeight`. Those calls show that `updateAssistantPanelGrowth` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `updateAssistantPanelGrowth`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

isDesktop: `isDesktop` stores a computed value for temporary calculation. It is initialized from `window.matchMedia("(min-width: 901px)").matches`. Within the function it is assigned or updated later in the function.

dockedHeight: `dockedHeight` stores a computed value for temporary calculation. It is initialized from `assistantDesktopDockedHeight()`. Within the function it is passed into `round`, `toggle`; assigned or updated later in the function.

formHeight: `formHeight` stores a computed value for temporary calculation. It is initialized from `aiAssistantForm.getBoundingClientRect().height || 46`. Within the function it is assigned or updated later in the function.

clearHeight: `clearHeight` stores a computed value for temporary calculation. It is initialized from `aiClearChatButton?.getBoundingClientRect().height || 38`. Within the function it is assigned or updated later in the function.

titleHeight: `titleHeight` stores a DOM element reference. It is initialized from `document.querySelector("#ai-assistant-title")?.getBoundingClientRect().height || 34`. Within the function it is assigned or updated later in the function.

chromeHeight: `chromeHeight` stores a computed value for temporary calculation. It is initialized from `titleHeight + formHeight + clearHeight + 92`. Within the function it is assigned or updated later in the function.

messages: `messages` stores a list for temporary calculation. It is initialized from `[...aiAssistantLog.querySelectorAll(".ai-message")]`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `reduce`; assigned or updated later in the function.

messageGap: `messageGap` stores a computed value for temporary calculation. It is initialized from `messages.length > 1 ? (messages.length - 1) * 10 : 0`. Within the function it is assigned or updated later in the function.

messageHeight: `messageHeight` stores a function or callback value for temporary calculation. It is initialized from `messages.reduce((sum, message) => sum + message.getBoundingClientRect().height, 0) + messageGap`. Within the function it is assigned or updated later in the function.

contentHeight: `contentHeight` stores a computed value for temporary calculation. It is initialized from `messageHeight + chromeHeight`. Within the function it is passed into `round`; assigned or updated later in the function.

maxHeight: `maxHeight` stores a computed value for temporary calculation. It is initialized from `Math.min(window.innerHeight * 0.76, 720)`. Within the function it is passed into `round`; assigned or updated later in the function.

nextHeight: nextHeight stores a computed value for temporary calculation. It is initialized from `Math.round(Math.max(dockedHeight, Math.min(contentHeight, maxHeight)))`. Within the function it is passed into `setProperty`, `toggle`; assigned or updated later in the function.

function renderAssistantInline(value = "")

Purpose and intuition: `renderAssistantInline` turns saved data for `AssistantInline` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

Parameters and data it receives: `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with `renderInlineMath`. Those calls show that `renderAssistantInline` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `renderAssistantInline`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function renderAssistantAnswerContent(content = "")

Purpose and intuition: `renderAssistantAnswerContent` turns saved data for `AssistantAnswerContent` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

Parameters and data it receives: `content` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

What it calls or works with: It works with `renderAssistantInline`, `escapeHtml`. Those calls show that `renderAssistantAnswerContent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `renderAssistantAnswerContent`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

lines: lines stores a computed value for temporary calculation. It is initialized from `String(content || "").replace(/r\n?/g, "\n").split("\n")`. Within the function it is iterated or transformed as a collection; mutated as a collection or DOM container; accesses properties/methods `forEach`, `join`, `push`; passed into `push`; assigned or updated later in the function.

html: html stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `push`; assigned or updated later in the function.

bulletItems: bulletItems stores a list for temporary calculation. It is initialized from `[]`. Within the function it is iterated or transformed as a collection; mutated as a collection or DOM container; accesses properties/methods `length`, `map`, `push`; passed into `push`; assigned or updated later in the function.

codeBlock: codeBlock stores a computed value for temporary calculation. It is initialized from `null`. Within the function it is accesses properties/methods `language`, `lines`; passed into `escapeHtml`, `push`; assigned or updated later in the function.

flushBullets: flushBullets stores a function or callback value for temporary calculation. It is initialized from `() => {}`. Within the function it is assigned or updated later in the function.

flushCode: flushCode stores a function or callback value for temporary calculation. It is initialized from `() => {}`. Within the function it is assigned or updated later in the function.

language: language stores a computed value for compiler or language configuration. It is initialized from `codeBlock.language ? `data-language="${escapeHtml(codeBlock.language)}" : ""`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `escapeHtml`, `push`; assigned or updated later in the function.

trimmed: trimmed stores a computed value for temporary calculation. It is initialized from `line.trim()`. Within the function it is accesses properties/methods `match`, `slice`; passed into `push`; assigned or updated later in the function.

fence: fence stores a computed value for temporary calculation. It is initialized from `trimmed.match(/^```([a-z0-9_+.#-]*)s*$/i)`. Within the function it is assigned or updated later in the function.

bullet: bullet stores a computed value for temporary calculation. It is initialized from `trimmed.match(/^[-*\u2022]\s+(.)u)`. Within the function it is passed into `push`; assigned or updated later in the function.

function appendAssistantMessage(role, content, sources = [])

Purpose and intuition: `appendAssistantMessage` belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `role` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `content` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `sources` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Important body details: creates DOM elements instead of relying only on static HTML; writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

What it calls or works with: It works with `renderAssistantAnswerContent`, `assistantSourcesMarkup`, `renderMultilineInlineText`, `updateAssistantPanelGrowth`. Those calls show that `appendAssistantMessage` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `appendAssistantMessage`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

article: `article` stores a DOM element reference. It is initialized from `document.createElement("article")`. Within the function it is returned to the caller; accesses properties/methods `className`, `innerHTML`; passed into `append`, `createElement`; assigned or updated later in the function.

function scrollAssistantAnswerToStart(article)

Purpose and intuition: `scrollAssistantAnswerToStart` belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `article` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: Inside the body, it uses a timer to avoid blocking or to wait for another process. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: No other named helper from this file is detected inside the body. That means `scrollAssistantAnswerToStart` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `scrollAssistantAnswerToStart`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

scrollToAnswer: `scrollToAnswer` stores a function or callback value for temporary calculation. It is initialized from `() => {}`. Within the function it is passed into `requestAnimationFrame`, `setTimeout`; assigned or updated later in the function.

function appendAssistantPendingMessage()

Purpose and intuition: `appendAssistantPendingMessage` belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Important body details: toggles CSS classes to express UI state.

What it calls or works with: It works with `appendAssistantMessage`. Those calls show that `appendAssistantPendingMessage` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `appendAssistantPendingMessage`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

article: article stores a computed value for temporary calculation. It is initialized from `appendAssistantMessage("assistant", "Thinking", [])`. Within the function it is returned to the caller; assigned or updated later in the function.

function `replaceAssistantMessage(article, content, sources = [])`

Purpose and intuition: `replaceAssistantMessage` belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `article` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `content` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `sources` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

What it calls or works with: It works with `appendAssistantMessage`, `renderAssistantAnswerContent`, `assistantSourcesMarkup`, `updateAssistantPanelGrowth`, `scrollAssistantAnswerToStart`. Those calls show that `replaceAssistantMessage` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `replaceAssistantMessage`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function `setAssistantBusy(isBusy)`

Purpose and intuition: `setAssistantBusy` belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: `isBusy` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

What it calls or works with: No other named helper from this file is detected inside the body. That means `setAssistantBusy` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why the file needs it: Without `setAssistantBusy`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

submitButton: `submitButton` stores a DOM element reference. It is initialized from `aiAssistantForm?.querySelector('button[type="submit"]')`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods disabled; assigned or updated later in the function.

function clearAssistantChat()

Purpose and intuition: `clearAssistantChat` belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

What it does: Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Important body details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

What it calls or works with: It works with `setAssistantStatus`, `updateAssistantPanelGrowth`. Those calls show that `clearAssistantChat` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `clearAssistantChat`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: No local `const`, `let`, or `var` values were detected. The function mostly works from its parameters, module-level state, literals, or immediate return expressions.

function assistantRemoteTimeoutMs(question = "", context = {}, options = {})

Purpose and intuition: `assistantRemoteTimeoutMs` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Parameters and data it receives: `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `context` is a

structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. options is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

What it calls or works with: It works with normalize. Those calls show that assistantRemoteTimeoutMs is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without assistantRemoteTimeoutMs, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

clean: clean stores a computed value for temporary calculation. It is initialized from normalize(question). Within the function it is passed into test; assigned or updated later in the function.

sourceFetches: sourceFetches stores a computed value for temporary calculation. It is initialized from Array.isArray(context.sourceFetches) ? context.sourceFetches : []. Within the function it is accesses properties/methods some; passed into isArray; assigned or updated later in the function.

usesGitHub: usesGitHub stores a function or callback value for Git publishing and authorization state. It is initialized from sourceFetches.some((source) => String(source.kind || "").toLowerCase() === "github" || /github\.com\/i.test(source.url || "")). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is assigned or updated later in the function.

async function askRemoteAssistant(question, context, options = {})

Purpose and intuition: askRemoteAssistant belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

Parameters and data it receives: question is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. context is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. options is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work, calls a network resource and uses a timer to avoid blocking or to wait for another process. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Important body details: serializes structured data before sending or saving it; throws explicit errors when a required safety or compatibility condition fails; performs a fetch and therefore must handle network delay and failure.

What it calls or works with: It works with `assistantEndpoint`, `assistantRemoteTimeoutMs`. Those calls show that `askRemoteAssistant` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `askRemoteAssistant`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

endpoint: `endpoint` stores a computed value for network or routing value. It is initialized from `assistantEndpoint()`. Within the function it is passed into `fetch`; assigned or updated later in the function.

controller: `controller` stores a constructed object for temporary calculation. It is initialized from `new AbortController()`. Within the function it is accesses properties/methods `abort`, `signal`; passed into `fetch`; assigned or updated later in the function.

didTimeout: `didTimeout` stores a boolean value for temporary calculation. It is initialized from `false`. Within the function it is assigned or updated later in the function.

timer: `timer` stores a function or callback value for temporary calculation. It is initialized from `setTimeout(() => {`. Within the function it is passed into `clearTimeout`; assigned or updated later in the function.

response: `response` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `json`, `ok`; assigned or updated later in the function.

data: `data` stores the completed result of an awaited operation. It is initialized from `await response.json()`. Within the function it is accesses properties/methods `answer`, `message`, `model`, `usedWebSearch`; passed into `Boolean`, `String`; assigned or updated later in the function.

answer: `answer` stores a computed value for temporary calculation. It is initialized from `String(data.answer || data.message || "").trim()`. Within the function it is returned to the caller; passed into `String`; assigned or updated later in the function.

async function answerAssistantQuestion(question = "")

Purpose and intuition: `answerAssistantQuestion` is the public chatbot orchestrator. It receives the user's typed question, appends it to chat history, classifies the intent, gathers relevant portfolio context, creates a pending assistant answer, calls the backend AI when appropriate, falls back to local answers when needed, renders sources only when relevant, and restores focus to the input.

Purpose and intuition: This function is `async` because the AI backend may need network time and because context gathering can include indexed file text. It is the reason `Ask My Portfolio` behaves like a conversation instead of a static search form.

Parameters and data it receives: `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

What it does: Inside the body, it waits for asynchronous work. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

What it calls or works with: It works with `appendAssistantMessage`, `assistantRemember`, `appendAssistantPendingMessage`, `setAssistantBusy`, `setAssistantStatus`, `assistantEndpoint`, `assistantQuestionIntent`, `assistantFallbackResults`. Those calls show that `answerAssistantQuestion` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why the file needs it: Without `answerAssistantQuestion`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Objects and variables inside this function: The following local values are important because they show how the function breaks the task into smaller pieces of state.

cleanQuestion: `cleanQuestion` stores a computed value for temporary calculation. It is initialized from `String(question || "").trim()`. Within the function it is passed into `appendAssistantMessage`, `askRemoteAssistant`, `assistantContextForQuestion`, `assistantConversationForRequest`, `assistantFallbackResults`, `assistantLocalAnswer`, `assistantQuestionIntent`, `assistantRemember`; assigned or updated later in the function.

priorConversation: `priorConversation` stores a computed value for temporary calculation. It is initialized from `assistantChatHistory.slice()`. Within the function it is passed into `assistantConversationForRequest`, `assistantQuestionIntent`; assigned or updated later in the function.

pendingMessage: `pendingMessage` stores a computed value for temporary calculation. It is initialized from `appendAssistantPendingMessage()`. Within the function it is passed into `replaceAssistantMessage`; assigned or updated later in the function.

intent: `intent` stores a computed value for temporary calculation. It is initialized from `assistantQuestionIntent(cleanQuestion, priorConversation)`. Within the function it is passed into `askRemoteAssistant`, `assistantContextForQuestion`, `assistantFallbackResults`, `assistantLocalAnswer`, `assistantSourcesForDisplay`; assigned or updated later in the function.

rawSources: `rawSources` stores a computed value for temporary calculation. It is initialized from `assistantFallbackResults(cleanQuestion, intent)`. Within the function it is passed into `assistantContextForQuestion`, `assistantSourcesForDisplay`; assigned or updated later in the function.

sources: `sources` stores a computed value for temporary calculation. It is initialized from `assistantSourcesForDisplay(cleanQuestion, rawSources, intent)`. Within the function it is passed into `assistantLocalAnswer`, `replaceAssistantMessage`; assigned or updated later in the function.

context: `context` stores a computed value for temporary calculation. It is initialized from `assistantContextForQuestion(cleanQuestion, intent, rawSources)`. Within the function it is passed into `askRemoteAssistant`; assigned or updated later in the function.

conversation: `conversation` stores a computed value for temporary calculation. It is initialized from `assistantConversationForRequest(cleanQuestion, priorConversation)`. Within the function it is assigned or updated later in the function.

remoteAnswer: `remoteAnswer` stores the completed result of an awaited operation. It is initialized from `await askRemoteAssistant(cleanQuestion, context, {`. Within the function it is accesses properties/methods `answer`,

model; passed into assistantRemember, replaceAssistantMessage, setAssistantStatus; assigned or updated later in the function.

fallbackAnswer: fallbackAnswer stores a computed value for temporary calculation. It is initialized from assistantLocalAnswer(cleanQuestion, sources, intent). Within the function it is passed into assistantRemember, replaceAssistantMessage; assigned or updated later in the function.

errorAnswer: errorAnswer stores a string value for temporary calculation. It is initialized from "I had trouble processing that question. Try rephrasing it with a project name, tool name, or electronics topic.". Within the function it is passed into assistantRemember, replaceAssistantMessage; assigned or updated later in the function.